On Linear Primitive Recursion*

Maria Emilia Maietti*, Valeria de Paiva†, Eike Ritter‡

*Dipartimento di Matematica Pura ed Applicata, Università di Padova, Italy.
†Xerox Palo Alto Research Center - USA.
‡School of Computer Science, University of Birmingham, UK.
maietti@math.unipd.it, E.Ritter@cs.bham.ac.uk, paiva@parc.xerox.com

Abstract

We extend the Dual Intuitionistic and Linear typed lambda calculus *DILL* with a Linear Natural Numbers Object (LNNO) and corresponding type-theoretical linear primitive recursion operator. This LNNO, unlike the previous notion of LNNO in the literature, does not satisfy weakening and contraction, thus it is semantically more linear than the previous proposal. We provide bounds for normalizing a term under substitutions, first for rudimentary intuitionistic linear lambda calculus (the fragment consisting of tensor and linear implication only) and then for the extension with the proposed Linear Natural Numbers towards a characterization of linear systems in terms of the complexity for normalizing their terms under substitution.

Contents

1 Introduction **2**

2 Linear Lambda Calculi **3**

3 On Linear Natural Numbers Objects **5**
3.1 Parè -Roman Linear Natural Numbers Object 5
3.2 The proposed Linear Natural Numbers Object 5
3.3 Hofmann's Linear Natural Numbers Object 8

4 Normalization of $DILL_{pr}$ and consequences **8**
4.1 Strong normalization and confluence of *DILLpr* 8
4.2 Absence of weakening and contraction 8
4.3 Representable functions 9

5 Complexity results **10**
5.1 Linear bound in pure intuitionistic linear lambda calculus 10
5.1.1 Virtual redexes 12
5.1.2 Why redefinition of type size 19
5.1.3 Counterexample with no commuting reductions 20
5.1.4 The pure implicational fragment 21
5.1.5 Extension with primitive recursion 21

6 Conclusion **23**

*Research supported by EPSRC-grant GR/L28296 under the title "The eXplicit Substitution Linear Abstract Machine" and done at the School of Computer Science, University of Birmingham, during employment of the first author as research fellow on the grant.

1 Introduction

Linear logic has been used to model optimizations for functional languages like update-in-place or optimal reductions. Recursion is an integral part of functional programming languages; in particular any inductively defined datatype like lists or trees gives rise to recursively defined functions.

The usual analysis of recursion in type theory is via PCF [Plo77], sometimes called the mother of all toy languages. PCF consists basically of the simply typed lambda-calculus together with natural numbers, booleans and most importantly a fixpoint operator, which provides general recursion. This type-theoretical understanding is reflected in the categorical notions of object of numerals, conditionals and categorical fixpoint operators within cartesian closed categories. In the absence of a fixpoint operator the categorical counterpart of natural numbers with iterators is a natural numbers object.

The addition of recursion to linear type theories is the subject of this paper. Thus our goal is to extend the linear lambda calculus, associated to Girard's intuitionistic linear logic, with "linear" natural numbers and (possibly) a "linear" fixpoint operator for dealing with general recursion.

The version of intuitionistic linear lambda calculus we shall consider is Barber and Plotkin's *DILL* (Dual Intuitionistic and Linear Logic [BP97]) where there are dual contexts consisting of linear and intuitionistic variables. We aim to extend the translation (also called decoration) of the intuitionistic lambda calculus into *DILL* (due to Girard) to a decoration of the programming language *PCF* into a corresponding extension of *DILL* . Our ultimate goal is to use such a decoration to build an efficient abstract machine, which reduces intuitionistic terms once they have been automatically transformed into their linearly decorated versions. Since the pure linear reductions (i.e. reduction of terms without !) are done linearly, the efficiency of the machine will depend on how many intuitionistic terms the decoration is able to embed as purely linear lambda terms. In other words we want to have as few !'s in the terms as possible, while preserving meaning. An obvious strategy is to consider the "linear" analogues of the categorical concepts above. This obvious strategy has been tried before: Thus we could just consider the extension of *DILL* with the following fixpoint operator defined by T. Braüner in [Bra94]

$$\frac{\Gamma \mid _ \vdash f : !A \multimap A}{\Gamma \mid _ \vdash Y(f) : A}$$

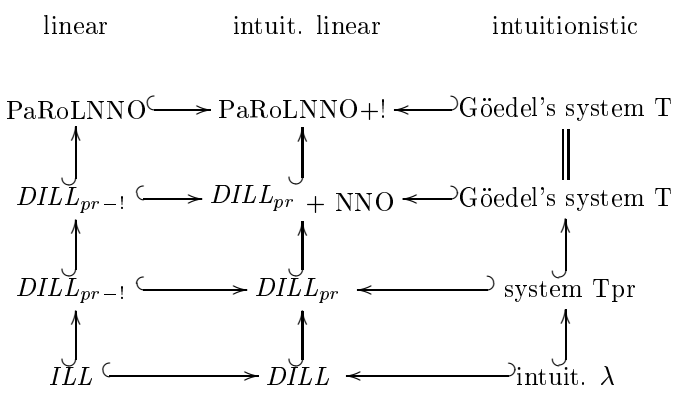
In this extension every function defined by recursion is translated as an intuitionistic one using Girard's decoration of intuitionistic logic into linear logic. Therefore with respect to functions defined by recursion, this decoration does not allow for improvement as far as reductions are concerned: all functions defined by recursion are totally intuitionistic. But some functions defined on the list data type, for example, "append" or "reverse", seem to be linear in their definitions and should not be decorated with the totally intuitionistic fixpoint above. Hence, we would like to find rules for recursion that enable us to detect as purely linear those *PCF* programs defined by recursion that do not use weakening nor contraction and have linear complexity.

Also the notion of a linear natural numbers object and of linear generalized primitive recursion schemata have been object of some study. Paré and Roman [PR89] proposed a natural numbers object in the context of symmetric monoidal closed categories, which is the usual natural numbers object definition when the cartesian product is substituted by a tensor. But this definition, despite being in a symmetric monoidal closed category, permits the definition of the diagonal map and of the weakening map for their linear natural numbers object. That turns out to be a comonoid. Moreover in the presence of the modality "!" this natural numbers object becomes a retraction of its !-version. All this means that from a semantical point of view this object can not be considered linear.

In this paper we propose a notion of Linear Natural Numbers Object (LNNO) that is not a comonoid. We only consider primitive recursion and define a linear natural numbers object as a list type on a linear singleton. We first introduce linear list types that enjoy a restricted form of elimination rule and then we introduce a linear singleton. The obtained system is called $DILL_{pr}$ in its version with the modalities ! and $DILL_{pr-!}$ without. The intuitionistic part of $DILL_{pr}$ corresponds to a restricted version of Gödel's system T where only non-parameterized primitive recursive functions are definable.

If we want to get system T in the intuitionistic part we need to make $DILL_{pr}$ more powerful with

an intuitionistic Natural Numbers Object (NNO). We get the following inclusions of theories:



where system Tpr is Göedel's system T with only non-parameterized primitive recursive functions and we mean *DILL* with Paré and Roman's LNNO with PaRoLNNO +! and the corresponding version without ! with PaRoLNNO.

The resulting LNNO, contrary to Paré and Roman's LNNO, does not satisfy contraction and weakening and let us write linear polynomials with only one recursion. All the definable linear functions on linear natural numbers are strictly monotone.

Thus far we have only considered a definition of "linear" natural numbers object from the semantical point of view: in other terms a linear natural numbers object must be definable in a symmetrical monoidal closed category and must not live in its category of comonoids. But in order to give a positive definition of when an object in a type theory is linear – apart from preventing the morphisms corresponding to the structural rules of weakening and contraction – we investigate the intuitionistic linear typed lambda calculus from a complexity point of view.

If we consider the intuitionistic linear typed lambda calculus without modalities, with equality given by β-reduction of these terms, and appropriate commuting conversions for tensor and its unit— a type theory we call $DILL_{-!}$ — we observe that we can provide a bound for the number of steps for normalizing a term under multiple substitutions. The reason for the existence of this bound is that, due to the lack of weakening and contraction, $DILL_{-!}$ satisfies a strong Church-Rosser property: if $t \rightsquigarrow t_1$ and $t \rightsquigarrow t_2$ in one step and $t_1 \neq t_2$, then there exists a term s such that $t_1 \rightsquigarrow s$ and $t_2 \rightsquigarrow s$ in one step. Clearly the strong Church-Rosser property fails in general in the presence of contraction. This bound states that the length of the longest reduction sequence of a term $t[s/x]$ is linear in the length of the longest reduction sequence of s. The proof can be obtained by induction on the size of the term and relies mostly (but not totally) on the strong Church-Rosser property mentioned. Although this bound is entirely plausible, its proof is non-trivial. Indeed, it is worth noticing that the above bound no longer holds if we consider only the β-reductions of tensor and its unit with no commuting conversions. The above bound confirms that any possible redex in any subterm can give rise to at most one redex in the whole term, namely each redex is reduced at most once during reduction.

More elaborate versions of these bounds for normalization under substitutions can also be shown both for the calculus with Paré and Roman's LNNO and that one with our Linear Natural Numbers Object. The corresponding result that no duplication of redexes for substitutable subterms is possible is then also confirmed for these systems. This means that sharing of arbitrary expressions during reduction is not definable even via the contraction map obtained from Paré and Roman's Linear Natural Numbers Object.

2 Linear Lambda Calculi

The linear type theory we consider is *DILL* , that stands for Dual Intuitionistic and Linear Logic and was introduced by Barber and Plotkin in [BP97]. This is a version with double contexts of Girard's Intuitionistic Linear Logic [Gir87], which was first decorated with a lambda calculus, called *ILL* , with usual single contexts in [BBdH93]. A proof of the equivalence between *DILL* and *ILL* is shown in

[Bar97]. *DILL* borrows from Girard's Logic of Unity the elegant idea of separating assumptions into two classes: intuitionistic, which can be freely duplicated (shared) or discarded (ignored); and linear, which are constrained to be used exactly once. Syntactically, this strict separation is achieved by maintaining judgements with double-sided ("dual") contexts $\Gamma \mid \Delta \vdash A$, where, as a convention, Γ and Δ contain non-linear (intuitionistic) and linear assumptions, respectively. In the calculus an intuitionistic assumption $a : A$ corresponds to the linear assumption $x :!A$ by the rules acting on the modality !.

About semantics, the fragment of intuitionistic linear logic without the modality ! has an uncontroversial complete categorical semantics given by symmetric monoidal closed categories. A complete semantics for *DILL* is given by symmetric monoidal adjunctions [BP97]. Here, as semantics of *DILL* we consider only those symmetric monoidal adjunctions given by the embedding functor into the free product co-Kleisli category associated to a monoidal comonad in a symmetric monoidal closed category as presented in [Bie94, Ben95].

Now we recall the definition of *DILL* . The type theory *DILL* has judgements of the form $\Gamma \mid \Delta \vdash M : A$, where Γ consists of a set of intuitionistic variable declarations of the form $a\colon A$ where no a is repeated and Δ consists of a multiset of linear variable declarations of the form $x\colon A$, again no variable is repeated. The set of types we shall work with is

$$A ::= G \mid !A \mid A \multimap A \mid A \otimes B \mid I$$

where G consists of some ground types.

Raw terms are defined inductively by:

$$\begin{array}{rcl} m, m & ::= & x \mid \lambda x^A.m \mid m(n) \mid !m \mid \mathtt{let}\ m\ \mathtt{be}\ !a\ \mathtt{in}\ n \\ & & \mid m \otimes n \mid \mathtt{let}\ m\ \mathtt{be}\ x_1 \otimes x_2\ \mathtt{in}\ n \mid \bullet \mid \mathtt{let}\ m\ \mathtt{be}\ \bullet\ \mathtt{in}\ n \end{array}$$

The typing rules of *DILL* are given by:

$$\Gamma \mid x : A \vdash x : A \qquad \Gamma, a\colon A \mid \emptyset \vdash a\colon A$$

$$\frac{\Gamma \mid \Delta, x : A \vdash m : B}{\Gamma \mid \Delta \vdash \lambda x^A.m : A \multimap B} \qquad \frac{\Gamma \mid \Delta_1 \vdash m\colon A \multimap B \quad \Gamma \mid \Delta_2 \vdash n\colon A}{\Gamma \mid \Delta \vdash m(n)\colon B}$$

$$\frac{\Gamma \mid \Delta_1 \vdash m : A \quad \Gamma \mid \Delta_2 \vdash n : B}{\Gamma \mid \Delta \vdash m \otimes n : A \otimes B} \qquad \frac{\Gamma \mid \Delta_1 \vdash m : A \otimes B \quad \Gamma \mid \Delta_2, x : A, y : B \vdash n : C}{\Gamma \mid \Delta \vdash \mathtt{let}\ m\ \mathtt{be}\ x \otimes y\ \mathtt{in}\ n : B}$$

$$\frac{}{\Gamma \mid \emptyset \vdash \bullet : I} \qquad \frac{\Gamma \mid \Delta_1 \vdash m : I \quad \Gamma \mid \Delta_2 \vdash n : C}{\Gamma \mid \Delta \vdash \mathtt{let}\ m\ \mathtt{be}\ \bullet\ \mathtt{in}\ n : C}$$

$$\frac{}{\Gamma \mid \emptyset \vdash \bullet : I} \qquad \frac{\Gamma \mid \emptyset \vdash m : A}{\Gamma \mid \emptyset \vdash !m :!A} \qquad \frac{\Gamma \mid \Delta_1 \vdash m :!A \quad \Gamma, a\colon A \mid \Delta_2 \vdash n\colon B}{\Gamma \mid \Delta \vdash \mathtt{let}\ m\ \mathtt{be}\ !a\ \mathtt{in}\ n : B}$$

In those rules with multiple premises, Δ_1, Δ_2 are disjoint and Δ is a permutation of Δ_1, Δ_2.

Because the let-expressions behave so similarly we sometimes write $\mathtt{let}\ M\ \mathtt{be}\ p\ \mathtt{in}\ N$ to cover both, where p is either $x_1 \otimes x_2$ or $\bullet$ or $!x$.

We have three kinds of equations, β and η-equations and commuting conversions. The last kind of equations, familiar in the setting of linear lambda-calculi, arise due to the form of η-rules for the tensor product and its unit. For the presentation of these equations we use contexts-with-holes, written $C[_]$. They are given by the grammar

$$\begin{array}{rcl} C[_] & ::= & _ \mid \lambda z^A.C[_] \mid C[_](m) \mid m(C[_]) \\ & & \mid C[_] \otimes m \mid m \otimes C[_] \mid \mathtt{let}\ C[_]\ \mathtt{be}\ p\ \mathtt{in}\ n \mid \mathtt{let}\ m\ \mathtt{be}\ p\ \mathtt{in}\ C[_] \end{array}$$

Note that this definition implies that there is exactly one occurrence of the symbol $_$ in a context-with-hole $C[_]$. The term $C[m]$ denotes the replacement of $_$ in $C[_]$ by m with the possible capture of free variables. This capture is the difference between the replacement of $_$ and substitution for a free variable: If $C[_]$ is the context-with-hole $(\lambda z^A._)$, then $C[a] = \lambda z^A.z$, whereas $(\lambda z^A.b)[a/b] = \lambda c^A.z$.

The equations for *DILL* are:

β-equations:

$$\begin{aligned}(\lambda z\colon A.m)(n) &= m[n/z]\\ \texttt{let } m\otimes n \texttt{ be } z_1\otimes z_2 \texttt{ in } r &= r[m/z_1, n/z_2]\\ \texttt{let } \bullet \texttt{ be } \bullet \texttt{ in } m &= m\\ \texttt{let } !a \texttt{ be } !x \texttt{ in } n &= n[a/x]\end{aligned}$$

η-equations:

$$\begin{aligned}\lambda a : A.m(a) &= m\\ \texttt{let } m \texttt{ be } z_1\otimes z_2 \texttt{ in } m &= m\\ \texttt{let } m \texttt{ be } \bullet \texttt{ in } \bullet &= m\\ \texttt{let } t \texttt{ be } !x \texttt{ in } !x &= t\end{aligned}$$

Commuting conver sions:

$$\begin{aligned}\texttt{let } m \texttt{ be } * \texttt{ in } C[n] &= C[\texttt{let } m \texttt{ be } * \texttt{ in } n]\\ \texttt{let } m \texttt{ be } z_1\otimes z_2 \texttt{ in } C[n] &= C[\texttt{let } m \texttt{ be } z_1\otimes z_2 \texttt{ in } n]\\ \texttt{let } m \texttt{ be } !x \texttt{ in } C[n] &= C[\texttt{let } m \texttt{ be } !x \texttt{ in } n]\end{aligned}$$

provided that in the last conversion C does not bind or contain x free.

3 On Linear Natural Numbers Objects

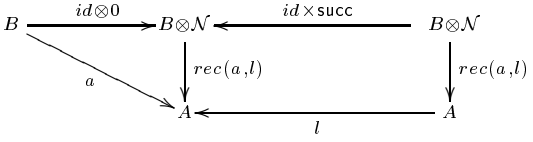
In order to tackle the problem of adding linear recursion to *DILL* we first address the problem of looking for a Linear Natural Numbers Object. To see our motivation for proceeding in this way, consider how a primitive recursive program is usually obtained in PCF:

$$\lambda n.\ rec(a,g,n)\ \equiv Y(\lambda f.\ \lambda n.\ \mathsf{If}\ iszero(n)\ \mathsf{then}\ a\ \mathsf{else}\ g(f(n))\ ):\ \mathcal{N}\to\mathcal{N}$$

Now, notice that we make use of the conditional operator If then else, which becomes an additive operator in a linear setting. But we can not think of an obvious way to turn the above use of conditional operator into an additive one. Hence, we decide to concentrate ourselves first on how to obtain at least primitive recursive programs in a linear setting and more precisely how to get a Linear Natural Numbers Object. After sorting out this in a future research we hope to explore the possibility of adding some kind of linear recursion. We could possibly start to look for computational aspects of the categorical notion of trace, since if the tensor is a product this corresponds to a fixpoint operator as proved in [Has99].

3.1 Parè -Roman Linear Natural Numbers Object

In the context of a symmetric monoidal closed category Paré and Roman in [PR89] introduced a Natural Numbers Object as follows: for all given $a : B \to A$ and $l : A \to A$ there exists a unique $rec(a,l)$ such that the following diagram commutes



This is obtained from the definition of Natural Numbers Object in a cartesian category just by substituting the product with tensor.

3.2 The proposed Linear Natural Numbers Object

In order to view Paré and Roman's Natural Numbers Object from a different perspective we consider to add the typing rules for the lists to the intuitionistic linear lambda calculus *DILL* :

List type

$$\text{list)}\ \frac{\vdash C\ type}{\vdash List(C)\ type}$$

$$\text{I}_1\text{-list)}\ \ \Delta \mid \vdash \epsilon : List(C) \qquad \text{I}_2\text{-list)}\ \ \Delta \mid\ s : List(C), c : C \vdash \mathsf{cons}(s,c) : List(C)$$

$$E_l\text{list)}\ \ \frac{\Delta \mid\ \Gamma \vdash a : A \qquad \Delta \mid\ x : A, y : C \vdash d(x,y) : A}{\Delta \mid\ \Gamma, s : List(C) \vdash \mathsf{Rec}(a,d,s) : A}$$

$$\text{C}_{1l}\text{-list)}\ \ \frac{\Delta \mid\ \Gamma \vdash a : A \qquad \Delta \mid\ x : A, y : C \vdash d(x,y) : A}{\Delta \mid\ \Gamma \vdash \mathsf{Rec}(a,d,\epsilon) = a : A}$$

$$\text{C}_{2l}\text{-list)}\ \ \frac{\Delta \mid\ \Gamma \vdash a : A \qquad \Delta \mid\ x : A, y : C \vdash d(x,y) : A}{\Delta \mid\ \Gamma, s : List(C), y : C \vdash \mathsf{Rec}(a,d,\mathsf{cons}(s,y)) = d(\mathsf{Rec}(a,d,s),y) : A}$$

We can see that the Natural Numbers Object in [PR89] is just isomorphic to $List(I)$, because $A \times I \simeq A$. This isomorphism agrees with the fact that the corresponding Natural Numbers Object in a cartesian closed category is indeed $List(Ter)$ where Ter is the terminal type. Here, we want to pursue this idea that the natural numbers are lists on a singleton.
It is easy to show that for Paré and Roman's natural numbers object $List(I)$ we can define a diagonal map

$$diag(x) \equiv \mathsf{Rec}(\epsilon \otimes \epsilon, \mathsf{cons} \otimes ((z)(\mathsf{cons}(z,\bullet)), s)$$

where $diag(x)\ : List(I) \otimes List(I)\ [x : List(I)]$ and also the semantical weakening

$$weak(x) \equiv \mathsf{Rec}(\bullet, \mathsf{id}, s)$$

where $weak(x)\ : I\ [x : List(I)]$ and $\mathsf{id} \equiv (x)x$. A closer analysis shows that the construction of a diagonal on $List(I)$ is due to the possibilities of having the following maps: the diagonal on I

$$I \to I \otimes I$$

and a distributive map

$$List(A \otimes B) \to List(A) \otimes List(B)$$

Hence, since lists are functorial, the diagonal results as the outcome of the following composition

$$List(I) \to List(I \otimes I) \to List(I) \otimes List(I)$$

In order to prevent a diagonal for the lists on a singleton, being this the intuitive representation of natural numbers, we can consider a type to be put in the place of I such that the duplicating map

$$I \to I \otimes I$$

is not allowed. This idea was developed by Hofmann in [Hof99] (see also sec.3.3). Here, we decide to forbid the construction of the second map

$$List(I \otimes I) \to List(I) \otimes List(I)$$

by keeping the idea that linear natural numbers are lists on a singleton that is proper inhabited.
Hence, we restrict the elimination rule on lists. Our linear list types are defined as follows:
Linear List type

$$\text{list}_l)\ \frac{\vdash C\ type}{\vdash List_l(C)\ type}$$

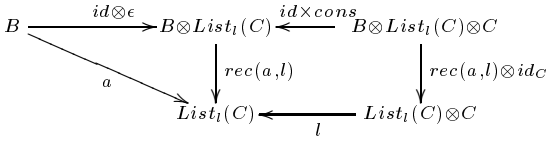
$$\text{I}_1\text{-list}_l)\ \ \Delta \mid \vdash \epsilon_l : List_l(C) \qquad \text{I}_2\text{-list}_l)\ \ \Delta \mid\ s : List_l(C), c : C \vdash \mathsf{cons}(s,c) : List_l(C)$$

$$E_l\text{list}_l)\ \ \frac{\Delta \mid\ \Gamma \vdash a : List_l(A) \qquad \Delta \mid\ x : List_l(C), y : C \vdash l(x,y) : List_l(C)}{\Delta \mid\ \Gamma, s : List_l(C) \vdash \mathsf{Rec}_{\mathsf{l}}(a,l,s) : List_l(C)}$$

$$\text{C}_{1l}\text{-list}_l)\ \ \frac{\Delta \mid\ \Gamma \vdash a : List_l(C) \qquad \Delta \mid\ x : List_l(C), y : C \vdash l(x,y) : List_l(C)}{\Delta \mid\ \Gamma \vdash \mathsf{Rec}_{\mathsf{l}}(a,l,\epsilon) = a : List_l(C)}$$

$$\mathrm{C}_{2l}\text{-list}_l)\quad \frac{\Delta \mid\ \Gamma \vdash a : List_l(C) \qquad \Delta \mid\ x : List_l(C), y : C \vdash l(x,y) : List_l(C)}{\Delta \mid\ \Gamma, s : List_l(C), y : C \vdash \mathsf{Rec}_{\mid}(a,l,\mathsf{cons}(s,y)) = l(\mathsf{Rec}_{\mid}(a,l,s),y) : List_l(C)}$$

whose categorical semantics should correspond to restricted parameterized list objects: for every a and l there is a unique $rec(a,l)$ such that the following diagram commutes



Due to the above restrictions in order to have back full lists in the intuitionistic side or at least a full intuitionistic natural numbers object we need an elimination rule for the intuitionistic case, since it does not come for free from the linear one:

List type

$$\text{list})\quad \frac{\mid \vdash C\ type}{\mid \vdash List(C)\ type}$$

$$\mathrm{I}_{i1}\text{-list})\ \ \Delta \mid \vdash \epsilon : List(C) \qquad \mathrm{I}_{i2}\text{-list})\ \ \Delta, s : List(C), c : C \mid \vdash \mathsf{cons}(s,c) : List(C)$$

$$E_i\text{-list})\quad \frac{\Delta \mid \vdash a : A \qquad \Delta, x : A, y : C \mid \vdash l(x,y) : A}{\Delta, s : List(C) \mid \vdash \mathsf{Rec}_{\mid}(a,l,s) : A}$$

$$\mathrm{C}_{1i}\text{-list})\quad \frac{\Delta \mid \vdash a : A \qquad \Delta, x : A, y : C \mid \vdash l(x,y) : A}{\Delta \mid \vdash \mathsf{Rec}_{\mid}(a,l,\epsilon) = a : A}$$

$$\mathrm{C}_{2i}\text{-list})\quad \frac{\Delta \mid \vdash a : A \qquad \Delta, x : A, y : C \mid \vdash l(x,y) : A}{\Delta, s \in List(C), y \in C \mid \vdash \mathsf{Rec}_{\mid}(a,l,\mathsf{cons}(s,y)) = l(\mathsf{Rec}_{\mid}(a,l,s),y) : A}$$

By this elimination rule the intuitionistic $List(C)$ will be interpreted as a parameterized list object.
Now we can represent the natural numbers object as the list on a linear singleton type.
But to represent the linear singleton we can not use the terminal type or the unit I, because they turn out to be isomorphic to their !-version in the presence of an elimination rule. Moreover, in both cases we can still define constant maps on these list types, namely for example

$$\overline{0}(x) \equiv \mathsf{Rec}_{\mid}(\epsilon, \phi, x)$$

where $\overline{0}(x)\ : List(I)\ [x : List(I)]$ and $\phi : List(I) \otimes I \to List(I)$ is the isomorphism related to tensor unit I.
Keeping in mind the above remark we add a weak form of singleton without an elimination rule

Singleton type

$$\mathrm{Tr})\ \ \top\ type \qquad \text{I-Tr})\ \ \Delta \mid \vdash \star : \top \qquad \text{C-Tr})\ \ \frac{\Delta \mid \Sigma\ \vdash t : \top \qquad \Delta \mid \Sigma\ \vdash t' : \top}{\Delta \mid \Sigma\ \vdash t = t' : \top}$$

In conclusion we define

$$\mathcal{N}^l \equiv List(\top)$$

as a candidate for a linear natural numbers object.
We call this extension $DILL_{pr}$.
Note that in $DILL$ in the presence of a falsum type defined as follows

False type

$$\mathrm{Fs})\ \ \bot\ type \qquad \text{E-Fs})\ \ \Delta \mid\ \Gamma, x : \bot\ \vdash\ r_o(y,x) : A \qquad \text{C-Fs})\ \ \frac{\Delta \mid \Gamma, x : \bot\ \vdash\ a : A}{\Delta \mid \Gamma, x : \bot\ \vdash\ a = r_o(y,x) : A}$$

interpreted as an initial object, we can consider $\bot \multimap 1$ as the singleton type. In $DILL$ $\bot \multimap 1$ does not imply its !-version by cut-elimination.

3.3 Hofmann's Linear Natural Numbers Object

In [Hof99] Martin Hofmann presents a linear natural numbers object in the context of affine linear logic, but it can be considered in the context of pure linear type theory. To define linear natural numbers he introduces a type $\diamond$ with *neither introduction nor elimination rules.* The natural numbers object is

$$List(\diamond)$$

and a general list type on a type A is

$$List(\diamond \otimes A)$$

that is you are allowed only to have lists type with a diamond.
But the important difference is that in the presence of $\diamond$ the linear elimination rule can be enforced to be

$$E_l\text{list)} \quad \frac{\Delta \mid \ \Gamma \vdash a : A \qquad \Delta \mid \ x : A, y : C, z : \diamond \vdash l(x,y,z) : A}{\Delta \mid \ \Gamma, s : List(C) \vdash \mathsf{Rec}_{\mathsf{l}}(a,l,s) : A}$$

without being able to define a contraction map. But the closed terms of a list are only the empty list

$$\epsilon : List(\diamond \otimes A)$$

Therefore the only closed natural numbers are zero! The number n can be represented as

$$x_1 : \diamond, \ldots, x_n : \diamond \vdash \mathsf{cons}(\mathsf{cons}(\mathsf{cons}(\ldots \mathsf{cons}(\epsilon, x_1), x_2), x_n) \in List(\diamond)$$

In the context of pure linear logic you can define the addition,

$$+ : List(\diamond) \otimes List(\diamond) \multimap List(\diamond)$$

and then by substitution also $x + n$ for $\overline{z} : \diamond^n \vdash n : List(\diamond)$. But it seems you can not write the multiplication for a constant like

$$2 \cdot x : List(\diamond) \multimap List(\diamond)$$

since in the recursive call you need to put cons twice.
Moreover, in the presence of the modality !, categorically speaking, an intuitionistic natural numbers object can be recovered only in the comma category over $!\diamond$ with these definitions.

4 Normalization of $DILL_{pr}$ and consequences

4.1 Strong normalization and confluence of *DILLpr*

A strong normalization theorem holds for the previous extension *DILLpr* with the usual method of logical relations (see for ex. [GTL89] and [Gha95]) by considering as reductions all the β reductions and suitable commuting conversions. Also the Church-Rosser property of confluence can be proved using the standard technique.

4.2 Absence of weakening and contraction

The semantical criteria to identify a linear object is stated in a negative way. It corresponds to proving that the object in consideration does not enjoy weakening and contraction. By strong normalization we can easily prove that our proposal of linear natural numbers object satisfies this linear semantical criteria.
We prove the following lemmas

Proposition 4.1 *For any $c(x) : C$ $[x : List(A)]$ derived in $DILL_{pr}$ and in normal form such that C is not $List(A)$ then C is not I and $c(x)$ is in introductory normal form.*

Proof. We proceed by induction on the size of the term. The statement can be easily deduced by using the inductive hypothesis. Indeed, $c(x)$ can not be in elimination form since it would not be in normal form and the case that $c(x)$ has rec as outmost constructor has been excluded by the restricted elimination rule for list types.

■

Now we are ready to conclude that there is no semantical diagonal and weakening.

Corollary 4.2 *The diagonal map, namely a term* $diag(x) : List(A) \otimes List(A)\ [x : List(A)]$ *such that for every* $n \in List(A)$ *closed then* $diag(n) = n \otimes n$*, is not definable in* $DILL_{pr}$.

Proof. We assume $diag(x)$ in normal form. By the above proposition it should be in introductory normal form, that is a tensor product. But, since the variable appears only once, one of its member should be fixed, against the semantics of the diagonal.

■

By the above proposition is immediate to deduce that:

Corollary 4.3 *The weakening map, namely a term* $weak(x) : I\ [x : List(A)]$ *such that for every* $n \in List(A)$ *closed then* $weak(n) = \bullet$*, is not definable in DILLpr.*

We also conclude very easily:

Proposition 4.4 *DILLpr is conservative over intuitionistic linear logic.*

To get this conservativity result the restriction on the elimination rule for lists is crucial.

4.3 Representable functions

With the previous definition of linear natural numbers and corresponding linear primitive recursion we can represent the linear functions with only one recursion, that is

Proposition 4.5 *For any fixed* $m \in N$ *we derive in* $DILL_{pr}$ *using only one recursion step*

$$\frac{\vdash a_o : \mathcal{N}^l \quad \vdash a_1 : \mathcal{N}^l \quad \ldots \quad \vdash a_m : \mathcal{N}^l}{x_1 : \mathcal{N}^l, \ldots x_m : \mathcal{N}^l \vdash a_o + a_1 \cdot x_1 + \ldots a_m \cdot x_m : \mathcal{N}^l}$$

Proof.
The addition on natural numbers

$$x : \mathcal{N}^l, y : \mathcal{N}^l \vdash \ x + y : \mathcal{N}^l$$

corresponds to the composition of lists and it is defined by

$$x + y \equiv rec(x, \mathsf{cons}, y)$$

The multiplication for a constant

$$\frac{\vdash a : \mathcal{N}^l}{x : \mathcal{N}^l \vdash \ a \cdot x : \mathcal{N}^l}$$

is defined once we have normalized a to some normal form n as

$$rec(\ \epsilon\ ,\ (x, y)\ \mathsf{cons}(x, y) + (n - 1)\ ,\ y\ )$$

where $x + n \equiv \mathsf{cons}(\mathsf{cons}(\ldots \mathsf{cons}(x, *), *), *)$ is the n-times composition of cons with $*$ in the second member.
Finally by a succession of cuts we get the above linear function. However, we could define it without cuts with the suitable one step recursion.

■

We can also derive the function *append* and *reverse* in the usual way with primitive recursion on lists.

Lemma 4.6 *Any* $t(y) : C\ [y : \top]$ *with* C *not* $\top$ *in normal form in* $DILL_{pr}$ *is in introductory normal form.*

Proof. The proof goes by induction on the size of $t(y)$. We deduce that $t(y)$ can not be in elimination normal form by using the inductive hypothesis and the fact that the term is in normal form.
■

Def. 4.7 *We define $Mis(m)$ for every closed term $m : List(\top)$ as follows: $Mis(\epsilon) = 0$ and $Mis(cons(a, c)) = Mis(a) + 1$. Then we write $m < n$ for $Mis(m) < Mis(n)$.*

Now we prove that

Proposition 4.8 *Any $t(x) : List(\top)$ $[x : List(\top)]$ derived in $DILL_{pr}$ and in normal form is strictly monotone, namely for every $n, m : List(\top)$ if $n < m$ then $t(n) < t(m)$ and $t(n) \geq n$. Moreover, for every $g(x, y) : List(\top)$ $[x : List(\top), y : \top]$ in normal form we have that $g(n, c) > n$.*

Proof. We proceed by induction using the fact that $t(x)$ is in normal form and proposition 4.1. The only case that needs a bit of calculation is when the outmost constructor of $t(x)$ is $\mathsf{rec}(a, g, b)$ where the variable x appears in b (recall that the variable x does not appear in g by the typing rule for lists) otherwise b is closed and $t(x)$ is not in normal form. First we consider the case $t(x) \equiv \mathsf{rec}(a, g, x)$. After reducing $\mathsf{rec}(a, g, n)$ and $\mathsf{rec}(a, g, m)$ to their normal form, the first inequality follows since $g(a, c) > a$ for any closed $a : List(\top)$ by inductive hypothesis and for the same reason also the second one is valid. Supposing $g(x, y) \equiv \mathsf{rec}(a, g, x)$ then y appears in a and by lemma 4.6 also the third inequality holds. If $t(x)$ is $\mathsf{rec}(a, g, b)$ and b is not a variable we conclude the first inequality by inductive hypothesis on b and on $\mathsf{rec}(a, g, x)$. The second inequality follows from the inductive hypothesis on $\mathsf{rec}(a, g, x)$ and b, from which we get $\mathsf{rec}(a, g, b[n/x]) \geq b[n/x] \geq n$. To prove the third inequality we observe by lemma 4.6 that x must appear in b. If also y appears in b we immediately conclude by inductive hypothesis. If y appears in a then $a = \mathsf{cons}(a', c)$ from lemma 4.6 and surely $\mathsf{rec}(a[*/y], g, b[n/x]) > b[n/x] \geq n$. The other cases where the outmost constructor is obtained by elimination rule are not possible by proposition 4.1 since the term is in normal form. This is the case also for $g(x, y)$ since proposition 4.1 can be proved for it as well.
■

From this we can conclude

Corollary 4.9 *In $DILL_{pr}$ we can not define a term $t(x) : List(\top)$ $[x : List(\top)]$ such that $t(n) = t(m)$ for $n < m$, $n, m : List(\top)$.*

With this corollary we exclude the possibility to write a constant map contrary to the case of taking $List(I)$ as a linear natural numbers even with our restricted elimination rule for lists.

5 Complexity results

After sorting out a notion of linear primitive recursion we are interested in studying the complexity of normalization in the obtained system by consider a reduction step as our complexity measure. The fact that linear variables appear only once should intuitively give us some bounds for the maximum number of steps to normalize a term under substitutions, of course in the absence of the modality !. But we will see that this is not entirely a consequence of linear occurrences: it also depends on the one step reductions we consider in the linear system.

5.1 Linear bound in pure intuitionistic linear lambda calculus

Our aim is to find a bound of the maximum number of steps to normalize linear terms after performing simultaneous substitutions. This bound should depend on the maximum number of steps to normalize the terms substituted. Since the linear variables appear only once in the purely linear lambda calculus (in absence of additives!) we expect that this bound is a linear combination of the maximum number of steps to normalize each term substituted. Indeed, we are going to prove this here. Our proof is by induction on the size of the term. To perform this induction we need to know exactly the coefficients of this linear combination. The coefficients are defined only on terms in normal form in such a way to predict the maximum number of redexes that a normal form produces after any bunch of substitutions.

$$a : A \vdash a : A$$

$$\frac{\Delta, z : A \vdash b : B}{\Gamma \mid \Delta \vdash \lambda z^A.b : A \multimap B} \qquad \frac{\Delta_1 \vdash f : A \multimap B \quad \Delta_2 \vdash a : A}{\Delta \vdash f(a) : B}$$

$$\frac{\Delta_1 \vdash a : A \quad \Delta_2 \vdash b : B}{\Delta \vdash a \otimes b : A \otimes B} \qquad \frac{\Delta_1 \vdash d : A \otimes B \quad z_1 : A, z_2 : B, \Delta_2 \vdash c : C}{\Delta \vdash \texttt{let}\ d\ \texttt{be}\ z_1 \otimes z_2\ \texttt{in}\ c : B}$$

$$\frac{}{\emptyset \vdash \bullet : I} \qquad \frac{\Delta_1 \vdash m : I \quad \Delta_2 \vdash c : C}{\Delta \vdash \texttt{let}\ m\ \texttt{be}\ \bullet\ \texttt{in}\ c : C}$$

Where applicable Δ_1, Δ_2 are disjoint and Δ is a permutation of Δ_1, Δ_2.

Table 1: The typing rules of $DILL_{-!}$

We work on $DILL_{-!}$, that is how we call the fragment of pure intuitionistic linear logic without modalities. $DILL_{-!}$ has got the following grammar of types and terms.
The set of types we shall work with is

$$A ::= G \mid A \multimap B \mid A \otimes B \mid I$$

The syntax of preterms is defined inductively by

$$\begin{array}{rcl} m, n & ::= & x \mid \lambda x^A.m \mid m(n) \mid m \otimes n \mid \bullet \\ & & \mid \texttt{let}\ m\ \texttt{be}\ a \otimes b\ \texttt{in}\ n \mid \texttt{let}\ m\ \texttt{be}\ \bullet\ \texttt{in}\ n \end{array}$$

The reduction rules we consider are the following:

$$\begin{array}{rcl} (\lambda z\colon A.m)(n) & \underset{1}{\longrightarrow} & m[n/z] \\ \texttt{let}\ m \otimes n\ \texttt{be}\ z_1 \otimes z_2\ \texttt{in}\ r & \underset{1}{\longrightarrow} & r[m/z_1, n/z_2] \\ \texttt{let}\ \bullet\ \texttt{be}\ \bullet\ \texttt{in}\ m & \underset{1}{\longrightarrow} & m \end{array}$$

Moreover we consider the following commuting reductions:

$$\begin{array}{rcl} (\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ f)(a) & \underset{1}{\longrightarrow} & \texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ f(a) \\ \texttt{let}\ (\texttt{let}\ w\ \texttt{be}\ p'\ \texttt{in}\ c)\ \texttt{be}\ p\ \texttt{in}\ f & \underset{1}{\longrightarrow} & \texttt{let}\ w\ \texttt{be}\ p'\ \texttt{in}\ (\texttt{let}\ c\ \texttt{be}\ p\ \texttt{in}\ f) \end{array}$$

With standard techniques we can prove a normalization theorem for such a calculus.

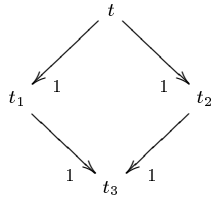
Def. 5.1 *Given a term t we define*

$$\nu(t) \equiv \textit{ the length of the longest reduction sequence of the strongly normalizing term } t$$

Def. 5.2 *For every term t in $DILL_{-!}$ $\texttt{nf}(t)$ is the normal form of t with respect to $\underset{1}{\longrightarrow}$.*

A characteristic of this linear lambda calculus is a strong version of the confluence property. Indeed the Church-Rosser property holds for one step reductions.

Proposition 5.3 *Given $t \underset{1}{\longrightarrow} t_1$, $t \underset{1}{\longrightarrow} t_2$ and $t_1 \neq t_2$ then there exists a term t_3 such that $t_1 \underset{1}{\longrightarrow} t_3$ and $t_2 \underset{1}{\longrightarrow} t_3$.*



Proof. The proof goes by induction on the size of t.
■

From this it easily follows that

Corollary 5.4 *For every t in ILLp, if $t \longrightarrow_1 t'$ then*

$$\nu(t) = 1 + \nu(t')$$

This fact that follows from the one step Church-Rosser property is the key point to establish a bound for the maximum number of steps to normalize terms under substitutions.

5.1.1 Virtual redexes

First we redefine the type size for technical purposes that we will be clear later.

Def. 5.5 *The size Sz of a type in DILL$_{-!}$ is defined as follows:*

$$Sz(I) = 2 \qquad Sz(A \multimap B) \equiv Sz(A) + Sz(B) + 2 \qquad Sz(A \otimes B) \equiv Sz(A) + Sz(B) + 2$$

Def. 5.6 *We define a coefficient to count the virtual redexes of a term related to a variable:*

$$V_x(x) \equiv 0 \qquad V_x(y) \equiv 0$$

$$V_x(a \otimes b) \equiv \begin{cases} V_x(a) & \text{if } x \text{ appears in } a \\ V_x(b) & \text{if } x \text{ appears in } b \end{cases}$$

$$V_x(\lambda y.b) \equiv V_x(b)$$

$$V_x(f(b)) \equiv \begin{cases} V_x(f) & \text{if } x \text{ appears in } f \text{ but not in head position} \\ V_x(b) & \text{if } x \text{ appears in } b \\ \Sigma_{i=1}^{n+1}(Sz(B_i)+2) & \text{if } f \equiv x(b_1)(b_2)\ldots(b_n),\ b \equiv b_{n+1} \\ & \text{where } b_i : B_i \text{ for } i = 1,\ldots,n+1. \end{cases}$$

$$V_x(\texttt{let } x \texttt{ be } z_1 \otimes z_2 \texttt{ in } h)) \equiv V_{z_1}(h) + V_{z_2}(h) + 2$$

$$V_x(\texttt{let } f(b) \texttt{ be } z_1 \otimes z_2 \texttt{ in } h)) \equiv \begin{cases} V_x(h) & \text{if } x \text{ appears in } h \\ V_x(f(b)) & \text{if } x \text{ appears in } f(b) \text{ but not in head position} \\ \Sigma_{i=1}^{n+1}(Sz(B_i)+2) + (V_{z_1}(h) + V_{z_2}(h) + 2) & \\ & \text{if } f \equiv x(b_1)(b_2)\ldots(b_n),\ b \equiv b_{n+1} \\ & \text{where } b_i : B_i \text{ for } i = 1,\ldots,n+1. \end{cases}$$

$$V_x(\texttt{let } x \texttt{ be } \bullet \texttt{ in } h) \equiv 2$$

$$V_x(\texttt{let } f(b) \texttt{ be } \bullet \texttt{ in } h) \equiv \begin{cases} V_x(h) & \text{if } x \text{ appears in } h \\ V_x(f(b)) & \text{if } x \text{ appears in } f(b) \text{ but not in head position} \\ \Sigma_{i=1}^{n+1}(Sz(B_i)+2)+2 & \text{if } f \equiv x(b_1)(b_2)\ldots(b_n),\ b \equiv b_{n+1} \\ & \text{where } b_i : B_i \text{ for } i = 1,\ldots,n+1. \end{cases}$$

Proposition 5.7 *For every variable $x : B$ appearing in a term f in normal form then*

$$V_x(f) \leq Sz(B)$$

Proof. The proof goes by induction on the size of t.
■

Lemma 5.8 *If f is a subterm of t, both in normal form, then*

$$V_x(f) \leq V_x(t)$$

for all x appearing at least in t.

Proof. The proof goes by induction on the size of t.
■

Proposition 5.9 *For every $f : A \multimap B$ in normal form but not introductory, if*

$$\mathsf{nf}(f[\overline{v}/\overline{x}]) = \lambda w.g$$

where $\overline{x} \equiv x_1, \ldots, x_n$ are the free variables appearing in f then one of the following case may happen

$$f \equiv \begin{cases} x_1 \\ x_i(b_1)\ldots(b_n) \\ \mathtt{let}\ x_i(b_1)\ldots(b_m)\ \mathtt{be}\ p\ \mathtt{in}\ \lambda w.g \\ \mathtt{let}\ x_i(b_1)\ldots(b_m)\ \mathtt{be}\ p\ \mathtt{in}\ x_k \\ \mathtt{let}\ x_i(b_1)\ldots(b_m)\ \mathtt{be}\ p\ \mathtt{in}\ x_k(a_1)\ldots(a_j) \end{cases}$$

where $1 \leq i \leq n$, $1 \leq k \leq n$, $i \neq k$ and possibly $m = 0$, that is no argument b_i appears.

Proposition 5.10 *For every $f : A \otimes B$ in normal form but not introductory, if*

$$\mathsf{nf}(f[\overline{v}/\overline{x}]) = a \otimes b$$

where $\overline{x} \equiv x_1, \ldots, x_n$ are the free variables appearing in f then one of the following case may happen

$$f \equiv \begin{cases} x_1 \\ x_i(b_1)\ldots(b_n) \\ \mathtt{let}\ x_i(b_1)\ldots(b_m)\ \mathtt{be}\ p\ \mathtt{in}\ a \otimes b \\ \mathtt{let}\ x_i(b_1)\ldots(b_m)\ \mathtt{be}\ p\ \mathtt{in}\ x_k \\ \mathtt{let}\ x_i(b_1)\ldots(b_m)\ \mathtt{be}\ p\ \mathtt{in}\ x_k(a_1)\ldots(a_j) \end{cases}$$

where $1 \leq i \leq n$, $1 \leq k \leq n$, $i \neq k$ and possibly $m = 0$, that is no argument b_i appears.

Proposition 5.11 *For every $f : I$ in normal form but not introductory, if*

$$\mathsf{nf}(f[\overline{v}/\overline{x}]) = \bullet$$

where $\overline{x} \equiv x_1, \ldots, x_n$ are the free variables appearing in f then one of the following case may happen

$$f \equiv \begin{cases} x_1 \\ x_i(b_1)\ldots(b_n) \\ \mathtt{let}\ x_i(b_1)\ldots(b_m)\ \mathtt{be}\ p\ \mathtt{in}\ \bullet \\ \mathtt{let}\ x_i(b_1)\ldots(b_m)\ \mathtt{be}\ p\ \mathtt{in}\ x_k \\ \mathtt{let}\ x_i(b_1)\ldots(b_m)\ \mathtt{be}\ p\ \mathtt{in}\ x_k(a_1)\ldots(a_j) \end{cases}$$

where $1 \leq i \leq n$, $1 \leq k \leq n$, $i \neq k$ and possibly $m = 0$, that is no argument b_i appears.

Proposition 5.12 *For every $t \equiv f(a)$ or $t \equiv \mathtt{let}\ f\ \mathtt{be}\ p\ \mathtt{in}\ d : D$ in normal form if*

$$\mathsf{nf}(f[\overline{v}/\overline{x}]) = \mathtt{let}\ c\ \mathtt{be}\ q\ \mathtt{in}\ h$$

then one of the following case happens:

$$f \equiv \begin{cases} x_1 \\ x_i(b_1)\ldots(b_n) \\ \mathtt{let}\ x_i(b_1)\ldots(b_m)\ \mathtt{be}\ p\ \mathtt{in}\ \bullet \\ \mathtt{let}\ x_i(b_1)\ldots(b_m)\ \mathtt{be}\ p\ \mathtt{in}\ x_k \\ \mathtt{let}\ x_i(b_1)\ldots(b_m)\ \mathtt{be}\ p\ \mathtt{in}\ x_k(a_1)\ldots(a_j) \quad \textit{for } 1 \leq i \leq n\ , 1 \leq j \leq n,\ i \neq k, \textit{ possibly } m = 0 \end{cases}$$

Corollary 5.13 *For every $f : A \multimap B$ in normal form but not introductory, $f(a) : B$ in normal form if*

$$\mathsf{nf}(f[\overline{v}/\overline{x}]) = \lambda w.g$$

where $\overline{x} \equiv x_1, \ldots, x_n$ are the free variables appearing in f then for some i

$$V_{x_i}(f(a)) \geq V_{x_i}(f) + 2 + Sz(A)$$

Corollary 5.14 *For every $f : I$ in normal form but not introductory, $\mathtt{let}\ f\ \mathtt{be}\ \bullet\ \mathtt{in}\ d : D$ in normal form if*

$$\mathsf{nf}(f[\overline{v}/\overline{x}]) = \bullet$$

where $\overline{x} \equiv x_1, \ldots, x_n$ are the free variables appearing in f then for some i

$$V_{x_i}(\mathtt{let}\ f\ \mathtt{be}\ \bullet\ \mathtt{in}\ d) \geq V_{x_i}(f) + 2$$

Corollary 5.15 *For every $f : A \otimes B$ in normal form but not introductory, $\mathtt{let}\ f\ \mathtt{be}\ z_1 \otimes z_2\ \mathtt{in}\ d : D$ in normal form if*

$$\mathsf{nf}(f[\overline{v}/\overline{x}]) = a \otimes b$$

where $\overline{x} \equiv x_1, \ldots, x_n$ are the free variables appearing in f then for some i

$$V_{x_i}(\mathtt{let}\ f\ \mathtt{be}\ z_1 \otimes z_2\ \mathtt{in}\ d) \geq V_{x_i}(f) + V_{z_1}(d) + V_{z_2}(d) + 2$$

Corollary 5.16 *For every $t \equiv f(a)$ or $t \equiv \mathtt{let}\ f\ \mathtt{be}\ p\ \mathtt{in}\ d : D$ in normal form if*

$$\mathsf{nf}(f[\overline{v}/\overline{x}]) = \mathtt{let}\ c\ \mathtt{be}\ q\ \mathtt{in}\ h$$

then one of the following case happens: for some i

$$\begin{cases} V_{x_i}(f(a)) \geq V_{x_i}(f) + 2 + Sz(A) & \text{if } t \equiv f(a) \\ V_{x_i}(\mathtt{let}\ f\ \mathtt{be}\ \bullet\ \mathtt{in}\ d) \geq V_{x_i}(f) + 2 & \text{if } t \equiv \mathtt{let}\ f\ \mathtt{be}\ \bullet\ \mathtt{in}\ d \\ V_{x_i}(\mathtt{let}\ f\ \mathtt{be}\ z_1 \otimes z_2\ \mathtt{in}\ d) \geq V_{x_i}(f) + V_{z_1}(b) + V_{z_2}(d) + 2 & \text{if } t \equiv \mathtt{let}\ f\ \mathtt{be}\ z_1 \otimes z_2\ \mathtt{in}\ d \end{cases}$$

Note that the size decreases during reductions:

Lemma 5.17 *If $t \xrightarrow[1]{} t'$ then*

$$size(t) \geq size(t')$$

On this we base part of the induction to prove the next theorems. The main theorem we want to prove provides a bound to normalize terms under substitutions.

Theorem 5.18 *For every term t in DILL$_{-!}$ where $\overline{x} \equiv x_1, \ldots, x_n$ are the free variables appearing in t then*

$$\nu(t[\overline{s}/\overline{x}]) \leq \Sigma_{i=0}^{n} \nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(t)) + \nu(t)$$

Proof. The proof goes by induction on $size(t[\overline{s}/\overline{x}])\omega + \nu(t)\omega^2$. Note that we abbreviate "inductive hypothesis" with ind. hyp.

1. If $t \equiv x_1$ the thesis is obviously valid.
2. If $t \equiv \lambda w.g$ then by definition of substitution and normal form

   $$\nu((\lambda w.g)[\overline{s}/\overline{x}]) = \nu(g([\overline{s}/\overline{x}])$$

   and by ind. hyp. on $size(g[\overline{s}/\overline{x}])$

   $$\nu(g([\overline{s}/\overline{x}]) \leq \Sigma_{i=0}^{n} \nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(g)) + \nu(g)$$

   and since $V_{x_i}(\mathsf{nf}(g)) = V_{x_i}(\mathsf{nf}(\lambda w.g))$ and $\nu(g) = \nu(\lambda w.g)$ we conclude.

3. If $t \equiv a \otimes b$ then

$$\nu(a \otimes b([\overline{s}/\overline{x}]) = \nu(a[\overline{s'}/\overline{x'}]) + \nu(b[\overline{s''}/\overline{x''}])$$

Then by ind. hyp. on $size(a[\overline{s'}/\overline{x'}])$

$$\nu(a[\overline{s'}/\overline{x'}]) \leq \Sigma_{i'=0}^{n}\nu(s'_{i'}) + \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(a)) + \nu(a)$$

and the same holds for b. Hence

$$\nu(a \otimes b[\overline{s}/\overline{x}]) \leq \Sigma_{i'=0}^{n_1}\nu(s_{i'}) + \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(a)) + \nu(a) + \Sigma_{i''=0}^{n_2}\nu(s_{i''}) + \Sigma_{i''=0}^{n_2} V_{x_{i''}}(\mathsf{nf}(b)) + \nu(b)$$

We then conclude since $\nu(a \otimes b) = \nu(a) + \nu(b)$, $V_{x'_{j'}}(\mathsf{nf}(a)) = V_{x'_{i'}}(\mathsf{nf}(a) \otimes \mathsf{nf}(b))$, $V_{x''_{i''}}(\mathsf{nf}(b)) = V_{x'_{i'''}}(\mathsf{nf}(a) \otimes \mathsf{nf}(b))$ and $\mathsf{nf}(a) \otimes \mathsf{nf}(b) = \mathsf{nf}(a \otimes b)$.

4. If $t \equiv f(a)$ with f not in normal form. Then if $f \underset{1}{\longrightarrow} f'$

$$\begin{array}{ll}
\nu(f(a)[\overline{s}/\overline{x}]) & = 1 + \nu(f'(a[\overline{s}/\overline{x}])) \\
& \quad \text{by cor.5.4} \\
& \leq 1 + \Sigma_{i=0}^{n}\nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(f'(a)) + \nu(f'(a)) \\
& \leq \Sigma_{i=0}^{n}\nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(f(a))) + \nu(f(a)) \\
& \quad \text{since } \nu(f(a)) = \nu(f'(a)) + 1 \text{ by cor.5.4 and } \mathsf{nf}(f(a)) = \mathsf{nf}(f'(a))
\end{array}$$

5. If $t \equiv f(a)$ with f in normal form but a not in normal form, considering $a \underset{1}{\longrightarrow} a'$ we proceed as in the previous case.

6. If $t \equiv f(a)$ with $f \equiv \lambda w.g$ in normal form and a in normal form then

$$\begin{array}{ll}
\nu((\lambda w.g)(a)[\overline{s}/\overline{x}]) & = 1 + \nu(g[a/w][\overline{s}/\overline{x}]) \\
& \quad \text{by cor.5.4} \\
& \leq 1 + \Sigma_{i=0}^{n}\nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(g[a/w])) + \nu(g[a/w]) \\
& \quad \text{by ind. hyp. on } size(g[a/w][\overline{s}/\overline{x}]) \\
& = \Sigma_{i=0}^{n}\nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}((\lambda w.g)(a))) + \nu((\lambda w.g)(a)) \\
& \quad \text{since by cor.5.4 } \nu((\lambda w.g)(a)) = 1 + \nu(g[a/w]) \\
& \qquad \text{and } \mathsf{nf}((\lambda w.g)(a)) = \mathsf{nf}(g[a/w])
\end{array}$$

7. If $t \equiv f(a)$ with $f \equiv \mathtt{let}\ c\ \mathtt{be}\ p\ \mathtt{in}\ d$ in normal form and a in normal form then

$$\begin{array}{ll}
\nu((\mathtt{let}\ c\ \mathtt{be}\ p\ \mathtt{in}\ d)(a)[\overline{s}/\overline{x}]) & = 1 + \nu(\mathtt{let}\ c\ \mathtt{be}\ p\ \mathtt{in}\ d(a)[\overline{s}/\overline{x}]) \\
& \quad \text{by cor.5.4} \\
& \leq 1 + \Sigma_{i=0}^{n}\nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(g[a/w])) + \nu(g[a/w]) \\
& \quad \text{by ind. hyp. since by cor.5.4} \\
& \qquad \nu(\mathtt{let}\ c\ \mathtt{be}\ p\ \mathtt{in}\ (d(a))) < \nu((\mathtt{let}\ c\ \mathtt{be}\ p\ \mathtt{in}\ d)(a)) \\
& = \Sigma_{i=0}^{n}\nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}((\lambda w.g)(a))) + \nu((\lambda w.g)(a)) \\
& \quad \text{since by cor.5.4} \\
& \qquad \nu((\mathtt{let}\ c\ \mathtt{be}\ p\ \mathtt{in}\ d)(a)) = 1 + \nu(\mathtt{let}\ c\ \mathtt{be}\ p\ \mathtt{in}\ d(a)) \\
& \qquad \text{and } \mathsf{nf}((\mathtt{let}\ c\ \mathtt{be}\ p\ \mathtt{in}\ d)(a)) = \mathsf{nf}(\mathtt{let}\ c\ \mathtt{be}\ p\ \mathtt{in}\ d(a))
\end{array}$$

8. If $t \equiv f(a)$ with $f(a)$ in normal form but at least a s_j is not in normal form, then if $s_j \underset{1}{\longrightarrow} s'_j$

$$\begin{array}{ll}
\nu(f(a)[\overline{s}/\overline{x}]) & = 1 + \nu(f(a)[s_i/x_i]_{i\neq j}[s'_j/x_j]) \\
& \quad \text{by cor.5.4} \\
& \leq \Sigma_{i=0,i\neq j}^{n}\nu(s_i) + \nu(s'_j)\Sigma_{i=0,i\neq j}^{n} V_{x_i}(\mathsf{nf}(f(a))) + \nu(f(a)) \\
& \quad \text{by ind. hyp. on } size(f(a)[s_i/x_i]_{i\neq j}[s'_j/x_j]) \\
& = \Sigma_{i=0}^{n}\nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(f(a))) + \nu(f(a)) \\
& \quad \text{since } \nu(s_j) = 1 + \nu(s'_j) \text{ by cor.5.4}
\end{array}$$

9. If $t \equiv f(a)$ with $f(a)$ and all s_i in normal form and $\mathsf{nf}(f[\overline{s'}/\overline{x'}]))$, where $\overline{x'}$ include only the free variables in f, is not introductory or a let-expression then by cor.5.4

$$\begin{aligned}\nu(f(a)[\overline{s}/\overline{x}]) &= \nu((f[\overline{s'}/\overline{x'}])(a[\overline{s''}/\overline{x''}])) \\ &= \nu(f[\overline{s'}/\overline{x'}]) + \nu(a[\overline{s''}/\overline{x''}])\end{aligned}$$

Hence by ind. hyp. on $size(f[\overline{s'}/\overline{x'}])$

$$\nu(f[\overline{s'}/\overline{x'}]) \leq \Sigma_{i'=0}^{n_1}\nu(s'_{i'}) + \Sigma_{i'=0}^{n_1}V_{x_{i'}}(\mathsf{nf}(f)) + \nu(f)$$

and the same holds for a. Summing the two bounds we conclude, since $\nu(f(a)) = 0$ and by lemma 5.8 $V_{x'_{j'}}(f) \leq V_{x'_{i'}}(f(a))$ and $V_{x''_{i''}}(a) \leq V_{x'_{i''}}(f(a))$.

10. If $t \equiv f(a)$ with $f(a)$ and all s_i in normal form and $\mathsf{nf}(f[\overline{s'}/\overline{x'}])) = \lambda w.g$ then

$$\begin{aligned}\nu(f(a)[\overline{s}/\overline{x}]) &= \nu((f[\overline{s'}/\overline{x'}])(a[\overline{s''}/\overline{x''}])) \\ &= \nu(f[\overline{s'}/\overline{x'}]) + \nu(\lambda w.g[\overline{s''}/\overline{x''}]) \\ &\quad \text{by cor.5.4} \\ &\leq \Sigma_{i'=0}^{n_1}V_{x_{i'}}(\mathsf{nf}(f)) + 1 + \nu(g[a[\overline{s''}/\overline{x''}]/w]) \\ &\quad \text{by ind. hyp. on } size(f[\overline{s'}/\overline{x'}]) \text{ and by cor.5.4} \\ &\leq \Sigma_{i'=0}^{n_1}V_{x_{i'}}(\mathsf{nf}(f)) + 1 + V_w(g) + \nu(a[\overline{s''}/\overline{x''}]) \\ &\quad \text{by ind. hyp. on } size(g[a[\overline{s''}/\overline{x''}]/w]) \text{ since } \nu(g) = \nu(s_i) = 0 \\ &\leq \Sigma_{i'=0}^{n_1}V_{x_{i'}}(\mathsf{nf}(f)) + 1 + Sz(A) + \Sigma_{i''=0}^{n_2}V_{x_{i''}}(\mathsf{nf}(a)) \\ &\quad \text{by prop.5.7 and by ind. hyp. on } size(a[\overline{s''}/\overline{x''}]) \text{ since } \nu(a) = 0 \\ &\leq \Sigma_{i=0}^{n}V_{x_i}(\mathsf{nf}(f(a))) \\ &\quad \text{by lemma 5.8 and by cor.5.13 stating that for some } j \\ &\qquad V_{x_j}(f(a)) \geq V_{x_j}(f) + 2 + Sz(A)\end{aligned}$$

11. If $t \equiv f(a)$ with $f(a)$ and all s_i in normal form and $\mathsf{nf}(f[\overline{s'}/\overline{x'}])) = \texttt{let}\ c\ \texttt{be}\ p\ \texttt{in}\ d$ then

$$\begin{aligned}\nu(f(a)[\overline{s}/\overline{x}]) &= \nu((f[\overline{s'}/\overline{x'}])(a[\overline{s''}/\overline{x''}])) \\ &= \nu(f[\overline{s'}/\overline{x'}]) + \nu((\texttt{let}\ c\ \texttt{be}\ p\ \texttt{in}\ d)(a[\overline{s''}/\overline{x''}])) \\ &\quad \text{by cor.5.4} \\ &\leq \Sigma_{i'=0}^{n_1}V_{x_{i'}}(\mathsf{nf}(f)) + 1 + \nu(\texttt{let}\ c\ \texttt{be}\ p\ \texttt{in}\ d(a)[\overline{s''}/\overline{x''}]) \\ &\quad \text{by ind. hyp. on } size(f[\overline{s'}/\overline{x'}]) \text{ since } \nu(f) = \nu(s_i) = 0 \\ &= \Sigma_{i'=0}^{n_1}V_{x_{i'}}(\mathsf{nf}(f)) + 1 + \nu(d(a)[\overline{s''}/\overline{x''}]) \\ &\quad \text{since } \texttt{let}\ c\ \texttt{be}\ p\ \texttt{in}\ d \text{ is in normal form}\end{aligned}$$

Now there are two subcases provided that d is in introductory normal form or not.

If d is not introductory then (recalling that $\mathsf{nf}(f(a)) = f(a)$)

$$\begin{aligned}\nu(f(a)[\overline{s}/\overline{x}]) &\leq \Sigma_{i'=0}^{n_1}V_{x_{i'}}(\mathsf{nf}(f)) + 1 + \nu(d(a)[\overline{s''}/\overline{x''}]) \\ &= \Sigma_{i'=0}^{n_1}V_{x_{i'}}(\mathsf{nf}(f)) + 1 + \nu(a[\overline{s''}/\overline{x''}]) \\ &\quad \text{since } \nu(d(a)[\overline{s''}/\overline{x''}]) = \nu(d) + \nu(a[\overline{s''}/\overline{x''}]) \text{ and } \nu(d) = 0 \\ &\leq \Sigma_{i'=0}^{n_1}V_{x_{i'}}(\mathsf{nf}(f)) + 1 + \Sigma_{i''=0}^{n_2}V_{x_{i''}}(\mathsf{nf}(a)) \\ &\quad \text{by ind.hyp on } size(a[\overline{s''}/\overline{x''}]) \text{ and } a \text{ in normal form} \\ &\leq \Sigma_{i=0}^{n}V_{x_i}(\mathsf{nf}(f(a))) \\ &\quad \text{by lemma 5.8 and by cor.5.16} \\ &\qquad \text{stating that for some } k\ V_{x_k}(f(a)) \geq 2 + V_{x_k}(f)\end{aligned}$$

If $d \equiv \lambda w.g$ then we proceed as follows:

$$\begin{aligned}
\nu(f(a)[\overline{s}/\overline{x}]) &\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(f)) + 1 + \nu((\lambda w.g)(a)[\overline{s''}/\overline{x''}]) \\
&\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(f)) + 1 + 1 + \nu(g[a[\overline{s''}/\overline{x''}]/w]) \\
&\quad \text{by cor.5.4} \\
&\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(f)) + 2 + V_w(g) + \nu(a[\overline{s''}/\overline{x''}]) \\
&\quad \text{by ind. hyp. on } size(g[a[\overline{s''}/\overline{x''}]/w]) \text{ since } \nu(g) = \nu(s_i) = 0 \\
&\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(f)) + 2 + Sz(A) + \Sigma_{i''=0}^{n_2} V_{x_{i''}}(\mathsf{nf}(a)) \\
&\quad \text{by prop.5.7 and by ind. hyp. on } size(a[\overline{s''}/\overline{x''}]), \text{ since } \nu(a) = 0 \\
&\leq \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(f(a))) \\
&\quad \text{since by cor.5.13 for some } j\ V_{x_j}(f(a)) \geq V_{x_j}(f) + 2 + Sz(A)
\end{aligned}$$

12. If $t \equiv \mathtt{let}\ w\ \mathtt{be}\ p\ \mathtt{in}\ h$ and $w \underset{1}{\longrightarrow} w'$ then

$$\begin{aligned}
\nu(\mathtt{let}\ w\ \mathtt{be}\ p\ \mathtt{in}\ h[\overline{s}/\overline{x}]) &= 1 + \nu(\mathtt{let}\ w'\ \mathtt{be}\ p\ \mathtt{in}\ h[\overline{s}/\overline{x}]) \\
&\quad \text{by cor.5.4} \\
&\leq 1 + \Sigma_{i=0}^{n} \nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(\mathtt{let}\ w'\ \mathtt{be}\ p\ \mathtt{in}\ h) + \nu(\mathtt{let}\ w'\ \mathtt{be}\ p\ \mathtt{in}\ h) \\
&\quad \text{by ind. hyp. on } size(\mathtt{let}\ w'\ \mathtt{be}\ p\ \mathtt{in}\ h[\overline{s}/\overline{x}]) \\
&\leq \Sigma_{i=0}^{n} \nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(\mathtt{let}\ w\ \mathtt{be}\ p\ \mathtt{in}\ h)) + \nu(\mathtt{let}\ w\ \mathtt{be}\ p\ \mathtt{in}\ h) \\
&\quad \text{since } \nu(\mathtt{let}\ w\ \mathtt{be}\ p\ \mathtt{in}\ h) = \nu(\mathtt{let}\ w'\ \mathtt{be}\ p\ \mathtt{in}\ h) + 1 \text{ by cor.5.4} \\
&\quad \text{and } \mathsf{nf}(\mathtt{let}\ w\ \mathtt{be}\ p\ \mathtt{in}\ h) = \mathsf{nf}(\mathtt{let}\ w'\ \mathtt{be}\ p\ \mathtt{in}\ h)
\end{aligned}$$

13. If $t \equiv \mathtt{let}\ w\ \mathtt{be}\ p\ \mathtt{in}\ h$ with w in normal form but h not in normal form, considering $h \underset{1}{\longrightarrow} h'$ we proceed as in the previous case.

14. If $t \equiv \mathtt{let}\ w\ \mathtt{be}\ p\ \mathtt{in}\ h$ with h in normal form and $w \equiv \bullet$ and $p \equiv \bullet$ then

$$\begin{aligned}
\nu(\mathtt{let}\ \bullet\ \mathtt{be}\ \bullet\ \mathtt{in}\ h[\overline{s}/\overline{x}]) &= 1 + \nu(h[\overline{s}/\overline{x}]) \\
&\quad \text{by cor.5.4} \\
&\leq 1 + \Sigma_{i=0}^{n} \nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(h)) + \nu(h) \\
&\quad \text{by ind. hyp. on } size(h[\overline{s}/\overline{x}]) \\
&= \Sigma_{i=0}^{n} \nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(\mathtt{let}\ \bullet\ \mathtt{be}\ \bullet\ \mathtt{in}\ h)) + \nu(\mathtt{let}\ \bullet\ \mathtt{be}\ \bullet\ \mathtt{in}\ h) \\
&\quad \text{since by cor.5.4 } \nu(\mathtt{let}\ \bullet\ \mathtt{be}\ \bullet\ \mathtt{in}\ h) = 1 + \nu(h) \\
&\quad \text{and } \mathsf{nf}(\mathtt{let}\ \bullet\ \mathtt{be}\ \bullet\ \mathtt{in}\ h) = \mathsf{nf}(h)
\end{aligned}$$

15. If $t \equiv \mathtt{let}\ w\ \mathtt{be}\ p\ \mathtt{in}\ h$ with h, w in normal form and $w \equiv a \otimes b$ and $p \equiv z_1 \otimes z_2$ then

$$\begin{aligned}
\nu(\mathtt{let}\ a \otimes b\ \mathtt{be}\ z_1 \otimes z_2\ \mathtt{in}\ h[\overline{s}/\overline{x}]) &= 1 + \nu(h[a/z_1, b/z_2][\overline{s}/\overline{x}]) \\
&\quad \text{by cor.5.4} \\
&\leq 1 + \Sigma_{i=0}^{n} \nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(h[a/z_1, b/z_2])) + \nu(h[a/z_1, b/z_2]) \\
&\quad \text{by ind. hyp. on } size(h[a/z_1, b/z_2][\overline{s}/\overline{x}]) \\
&= \Sigma_{i=0}^{n} \nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(\mathtt{let}\ a \otimes b\ \mathtt{be}\ z_1 \otimes z_2\ \mathtt{in}\ h) + \\
&\qquad + \nu(\mathtt{let}\ a \otimes b\ \mathtt{be}\ z_1 \otimes z_2\ \mathtt{in}\ h) \\
&\quad \text{since by cor.5.4} \\
&\quad \nu(\mathtt{let}\ a \otimes b\ \mathtt{be}\ z_1 \otimes z_2\ \mathtt{in}\ h) = 1 + \nu(h[a/z_1, b/z_2]) \\
&\quad \text{and } \mathsf{nf}(\mathtt{let}\ a \otimes b\ \mathtt{be}\ z_1 \otimes z_2\ \mathtt{in}\ h) = \mathsf{nf}(h[a/z_1, b/z_2])
\end{aligned}$$

16. If $t \equiv \mathtt{let}\ w\ \mathtt{be}\ p\ \mathtt{in}\ h$ with h, w in normal form and $w \equiv \mathtt{let}\ c\ \mathtt{be}\ q\ \mathtt{in}\ d$ then
$\nu(\mathtt{let}\ (\mathtt{let}\ c\ \mathtt{be}\ q\ \mathtt{in}\ d)\ \mathtt{be}\ p\ \mathtt{in}\ h[\overline{s}/\overline{x}]) =$

$$\begin{aligned}
&= 1 + \nu(\mathtt{let}\ c\ \mathtt{be}\ q\ \mathtt{in}\ (\mathtt{let}\ d\ \mathtt{be}\ p\ \mathtt{in}\ h)[\overline{s}/\overline{x}]) \\
&\quad \text{by cor.5.4} \\
&\leq 1 + \Sigma_{i=0}^{n} \nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(\mathtt{let}\ c\ \mathtt{be}\ q\ \mathtt{in}\ (\mathtt{let}\ d\ \mathtt{be}\ p\ \mathtt{in}\ h))) + \\
&\qquad + \nu(\mathtt{let}\ c\ \mathtt{be}\ q\ \mathtt{in}\ (\mathtt{let}\ d\ \mathtt{be}\ p\ \mathtt{in}\ h)) \\
&\quad \text{by ind. hyp. on } size(\mathtt{let}\ c\ \mathtt{be}\ q\ \mathtt{in}\ (\mathtt{let}\ d\ \mathtt{be}\ p\ \mathtt{in}\ h)[\overline{s}/\overline{x}]) \\
&\leq \Sigma_{i=0}^{n} \nu(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(\mathtt{let}\ w\ \mathtt{be}\ p\ \mathtt{in}\ h)) + \nu(\mathtt{let}\ w\ \mathtt{be}\ p\ \mathtt{in}\ h) \\
&\quad \text{since by cor.5.4} \\
&\quad \nu(\mathtt{let}\ w\ \mathtt{be}\ p\ \mathtt{in}\ h) = \nu(\mathtt{let}\ c\ \mathtt{be}\ q\ \mathtt{in}\ (\mathtt{let}\ d\ \mathtt{be}\ p\ \mathtt{in}\ h)) + 1 \\
&\quad \text{and } \mathsf{nf}(\mathtt{let}\ w\ \mathtt{be}\ p\ \mathtt{in}\ h) = \mathsf{nf}(\mathtt{let}\ c\ \mathtt{be}\ q\ \mathtt{in}\ (\mathtt{let}\ d\ \mathtt{be}\ p\ \mathtt{in}\ h))
\end{aligned}$$

17. If $t \equiv \texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h$ is in normal form and for some i $s_i \xrightarrow[1]{} s_i'$ we proceed as in the corresponding case with $t \equiv f(a)$.

18. If $t \equiv \texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h$ in normal form, all s_i also in normal form, $\mathsf{nf}(w[\overline{s'}/\overline{x'}])$ is not a let-expression or an introductory normal form corresponding to p then

$$\begin{aligned}
\nu(\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h[\overline{s}/\overline{x}]) &= \nu(\texttt{let}\ w[\overline{s'}/\overline{x'}]\ \texttt{be}\ p\ \texttt{in}\ h[\overline{s''}/\overline{x''}]) \\
&= \nu(w[\overline{s'}/\overline{x'}]) + \nu(h[\overline{s''}/\overline{x''}]) \\
&\quad \text{by cor.5.4} \\
&\leq \Sigma_{i'=0}^{n_1}\nu(s_{i'}') + \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(w)) + \Sigma_{i''=0}^{n_2} V_{x_{i''}}(\mathsf{nf}(h)) \\
&\quad \text{by ind. hyp. on } size(w[\overline{s'}/\overline{x'}]) \text{ and } size(h[\overline{s''}/\overline{x''}]) \\
&\quad\quad \text{being } w, h, s_i \text{ in normal form} \\
&\leq \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h)) + \nu(\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h) \\
&\quad \text{by lemma 5.8}
\end{aligned}$$

19. If $t \equiv \texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h$ in normal form, all s_i also in normal form, $p \equiv \bullet$ and $\mathsf{nf}(w[\overline{s'}/\overline{x'}]) = \bullet$ then

$$\begin{aligned}
\nu(\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h[\overline{s}/\overline{x}]) &= \nu(\texttt{let}\ w[\overline{s'}/\overline{x'}]\ \texttt{be}\ p\ \texttt{in}\ h[\overline{s''}/\overline{x''}]) \\
&= \nu(w[\overline{s'}/\overline{x'}]) + \nu(\texttt{let}\ \bullet\ \texttt{be}\ \bullet\ \texttt{in}\ h[\overline{s''}/\overline{x''}]) \\
&\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(w)) + 1 + \nu(h[\overline{s''}/\overline{x''}]) \\
&\quad \text{by ind. hyp. on } size(w[\overline{s'}/\overline{x'}]) \text{ and cor.5.4} \\
&\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(w)) + 1 + \Sigma_{i''=0}^{n_2} V_{x_{i''}}(\mathsf{nf}(h)) \\
&\quad \text{by ind. hyp. on } size(h[\overline{s''}/\overline{x''}]) \\
&\leq \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h)) \\
&\quad \text{by lemma 5.8}
\end{aligned}$$

20. If $t \equiv \texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h$ in normal form, all s_i also in normal form, $p \equiv z_1 \otimes z_2 : A \otimes B$ and $\mathsf{nf}(w[\overline{s'}/\overline{x'}]) = a \otimes b$ then

$$\begin{aligned}
\nu(\texttt{let}\ w\ \texttt{be}\ z_1 \otimes z_2\ \texttt{in}\ h[\overline{s}/\overline{x}]) &= \nu(\texttt{let}\ w[\overline{s'}/\overline{x'}]\ \texttt{be}\ z_1 \otimes z_2\ \texttt{in}\ h[\overline{s''}/\overline{x''}]) \\
&= \nu(w[\overline{s'}/\overline{x'}]) + \nu(\texttt{let}\ a \otimes b\ \texttt{be}\ z_1 \otimes z_2\ \texttt{in}\ h[\overline{s''}/\overline{x''}]) \\
&\quad \text{by cor.5.4} \\
&\leq +\Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(w)) + 1 + \nu(h[a/z_1, b/z_2][\overline{s''}/\overline{x''}]) \\
&\quad \text{by ind. hyp. on } size(w[\overline{s'}/\overline{x'}]) \text{ and by cor.5.4} \\
&\leq +\Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(w)) + 1 + \Sigma_{i''=0}^{n_2} V_{x_{i''}}(\mathsf{nf}(h)) + V_{z_1}(h) + V_{z_2}(h) \\
&\quad \text{by ind. hyp. on } size(h[a/z_1, b/z_2][\overline{s''}/\overline{x''}]) \\
&\quad\quad \text{since } \nu(h) = \nu(a) = \nu(b) = 0 \\
&\leq \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h)) \\
&\quad \text{by lemma 5.8 and by cor.5.15 stating that for some } j \\
&\quad\quad V_{x_j}(\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h) \geq 2 + V_{z_1}(h) + V_{z_2}(h) + V_{x_j}(w)
\end{aligned}$$

21. If $t \equiv \texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h$ in normal form, all s_i also in normal form, $\mathsf{nf}(w[\overline{s'}/\overline{x'}]) = \texttt{let}\ c\ \texttt{be}\ q\ \texttt{in}\ d$ and d not in an introductory normal form corresponding to p then

$$\begin{aligned}
\nu(\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h[\overline{s}/\overline{x}]) &= \nu(\texttt{let}\ w[\overline{s'}/\overline{x'}]\ \texttt{be}\ p\ \texttt{in}\ h[\overline{s''}/\overline{x''}]) \\
&= \nu(w[\overline{s'}/\overline{x'}]) + \nu(\texttt{let}\ (\texttt{let}\ c\ \texttt{be}\ q\ \texttt{in}\ d)\ \texttt{be}\ p\ \texttt{in}\ h[\overline{s''}/\overline{x''}]) \\
&\quad \text{by cor.5.4} \\
&\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(w)) + 1 + \nu(\texttt{let}\ c\ \texttt{be}\ q\ \texttt{in}\ (\texttt{let}\ d\ \texttt{be}\ p\ \texttt{in}\ h[\overline{s''}/\overline{x''}])) \\
&\quad \text{by ind. hyp. on } size(w[\overline{s'}/\overline{x'}]) \text{ and cor.5.4} \\
&= \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(w)) + 1 + \nu(h[\overline{s''}/\overline{x''}]) \\
&\quad \text{by assumption the let-expressions are in normal form} \\
&\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(w)) + 1 + \Sigma_{i''=0}^{n_2} V_{x_{i''}}(\mathsf{nf}(h)) \\
&\quad \text{by ind. hyp. on } size(h[\overline{s''}/\overline{x''}]) \text{ being } w, h, s_i \text{ in normal form} \\
&\leq \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h)) \\
&\quad \text{by lemma 5.8 and by cor.5.16} \\
&\quad \text{stating that for some } j\ V_{x_j}(\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h) \geq 2 + V_{x_j}(w)
\end{aligned}$$

22. If $t \equiv \texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h$ in normal form, all s_i also in normal form, $\mathsf{nf}(w[\overline{s'}/\overline{x'}]) = \texttt{let}\ c\ \texttt{be}\ q\ \texttt{in}\ d$ and $d \equiv \bullet$ and $p \equiv \bullet$ then

$$\begin{aligned}
\nu(\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h[\overline{s}/\overline{x}]) &= \nu(\texttt{let}\ w[\overline{s'}/\overline{x'}]\ \texttt{be}\ p\ \texttt{in}\ h[\overline{s''}/\overline{x''}]) \\
&= \nu(w[\overline{s'}/\overline{x'}]) + \nu(\texttt{let}\ (\texttt{let}\ c\ \texttt{be}\ q\ \texttt{in}\ \bullet)\ \texttt{be}\ \bullet\ \texttt{in}\ h[\overline{s''}/\overline{x''}]) \\
&\quad \text{by cor.5.4} \\
&\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(w)) + 1 + \nu(\texttt{let}\ c\ \texttt{be}\ q\ \texttt{in}\ (\texttt{let}\ \bullet\ \texttt{be}\ \bullet\ \texttt{in}\ h[\overline{s''}/\overline{x''}])) \\
&\quad \text{by ind. hyp. on } size(w[\overline{s'}/\overline{x'}]) \\
&= \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(w)) + 1 + 1 + \nu(\texttt{let}\ c\ \texttt{be}\ q\ \texttt{in}\ h[\overline{s''}/\overline{x''}]) \\
&\quad \text{by cor.5.4} \\
&= \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(w)) + 2 + \nu(h[\overline{s''}/\overline{x''}]) \\
&\quad \text{since } \texttt{let}\ c\ \texttt{be}\ q\ \texttt{in}\ h \text{ is in normal form} \\
&\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(w)) + 2 + \Sigma_{i''=0}^{n_2} V_{x_{i''}}(\mathsf{nf}(h)) \\
&\quad \text{by ind. hyp. on } size(h[\overline{s''}/\overline{x''}]) \\
&\leq \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h)) \\
&\quad \text{by lemma 5.8 and by cor.5.16 stating that for some } j \\
&\quad V_{x_j}(\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h) \geq 2 + V_{x_j}(w)
\end{aligned}$$

23. If $t \equiv \texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h$ in normal form, all s_i also in normal form, $\mathsf{nf}(w[\overline{s'}/\overline{x'}]) = \texttt{let}\ c\ \texttt{be}\ q\ \texttt{in}\ d$, $d \equiv a \otimes b$ and $p \equiv z_1 \otimes z_2 : A \otimes B$ then

$$\begin{aligned}
\nu(\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h[\overline{s}/\overline{x}]) &= \nu(\texttt{let}\ w[\overline{s'}/\overline{x'}]\ \texttt{be}\ p\ \texttt{in}\ h[\overline{s''}/\overline{x''}]) \\
&= \nu(w[\overline{s'}/\overline{x'}]) + \nu(\texttt{let}\ (\texttt{let}\ c\ \texttt{be}\ q\ \texttt{in}\ a \otimes b)\ \texttt{be}\ z_1 \otimes z_2\ \texttt{in}\ h[\overline{s''}/\overline{x''}]) \\
&\quad \text{by cor.5.4} \\
&\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(w)) + 1 + \\
&\qquad + \nu(\texttt{let}\ c\ \texttt{be}\ q\ \texttt{in}\ (\texttt{let}\ a \otimes b\ \texttt{be}\ z_1 \otimes z_2\ \texttt{in}\ h[\overline{s''}/\overline{x''}])) \\
&\quad \text{by ind. hyp. on } size(w[\overline{s'}/\overline{x'}]) \text{ and by cor.5.4} \\
&= \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(w)) + 1 + \nu(\texttt{let}\ a \otimes b\ \texttt{be}\ z_1 \otimes z_2\ \texttt{in}\ h[\overline{s''}/\overline{x''}]) \\
&\quad \text{being } \texttt{let}\ c\ \texttt{be}\ q\ \texttt{in}\ d \text{ in normal form} \\
&\leq + \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(w)) + 1 + 1 + \nu(h[a/z_1, b/z_2][\overline{s''}/\overline{x''}]) \\
&\quad \text{by cor.5.4} \\
&\leq + \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\mathsf{nf}(w)) + 2 + \Sigma_{i''=0}^{n_2} V_{x_{i''}}(\mathsf{nf}(h)) + V_{z_1}(h) + V_{z_2}(h) \\
&\quad \text{by ind. hyp. on } size(h[a/z_1, b/z_2][\overline{s''}/\overline{x''}]) \\
&\quad \text{since } \nu(h) = \nu(a) = \nu(b) = 0 \\
&\leq \Sigma_{i=0}^{n} V_{x_i}(\mathsf{nf}(\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h)) \\
&\quad \text{by lemma 5.8 and by cor.5.16 stating that for some } j \\
&\quad V_{x_j}(\texttt{let}\ w\ \texttt{be}\ p\ \texttt{in}\ h) \geq 2 + V_{z_1}(h) + V_{z_2}(h) + V_{x_j}(w)
\end{aligned}$$

■

5.1.2 Why redefinition of type size

We want to justify why we had to change the usual notion of the type size in order to get the theorem which establishes a bound for normalizing a term under substitutions.

- Why we put $Sz(I) = 2$.

  If you take $t \equiv \texttt{let}\ w\ \texttt{be}\ \bullet\ \texttt{in}\ h$ for any variable h then

$$\begin{aligned}
\nu(t[\texttt{let}\ x\ \texttt{be}\ \bullet\ \texttt{in}\ \bullet\ /w]) &= \nu(\texttt{let}\ (\texttt{let}\ x\ \texttt{be}\ \bullet\ \texttt{in}\ \bullet)\ \texttt{be}\ \bullet\ \texttt{in}\ h) \\
&= 1 + \nu(\texttt{let}\ x\ \texttt{be}\ \bullet\ \texttt{in}\ (\texttt{let}\ \bullet\ \texttt{be}\ \bullet\ \texttt{in}\ h)) \\
&\quad \text{by cor.5.4} \\
&= 1 + 1 + \nu(\texttt{let}\ x\ \texttt{be}\ \bullet\ \texttt{in}\ h) = 2
\end{aligned}$$

  Hence $V_w(t) = 2$. But since we want that $V_w(t) \leq Sz(I)$ we are forced to put $Sz(I) = 2$.

- Why we put $Sz(A \multimap B) \equiv Sz(A) + Sz(B) + 2$.

  If you take $t \equiv x(y)$ with $x : I \multimap I$ then

$$\begin{aligned}
&\nu(t[\texttt{let } z \texttt{ be } \bullet \texttt{ in } (\lambda w.\texttt{let } w \texttt{ be } \bullet \texttt{ in } \bullet)/x, \texttt{let } c \texttt{ be } \bullet \texttt{ in } \bullet\,/y]) = \\
&\quad = \nu((\texttt{let } z \texttt{ be } \bullet \texttt{ in } (\lambda w.\texttt{let } w \texttt{ be } \bullet \texttt{ in } \bullet))(\texttt{let } c \texttt{ be } \bullet \texttt{ in } \bullet)) \\
&\quad = 1 + \nu(\texttt{let } z \texttt{ be } \bullet \texttt{ in } ((\lambda w.\texttt{let } w \texttt{ be } \bullet \texttt{ in } \bullet)(\texttt{let } c \texttt{ be } \bullet \texttt{ in } \bullet))) \\
&\quad = 1 + 1 + \nu(\texttt{let } z \texttt{ be } \bullet \texttt{ in let } (\texttt{let } c \texttt{ be } \bullet \texttt{ in } \bullet) \texttt{ be } \bullet \texttt{ in } \bullet) \\
&\quad = 2 + 1 + \nu(\texttt{let } z \texttt{ be } \bullet \texttt{ in let } c \texttt{ be } \bullet \texttt{ in let } \bullet \texttt{ be } \bullet \texttt{ in } \bullet) \\
&\quad = 3 + 1 + \nu(\texttt{let } z \texttt{ be } \bullet \texttt{ in let } c \texttt{ be } \bullet \texttt{ in } \bullet) = 4
\end{aligned}$$

  Hence $V_x(x(y)) = 4 = 2 + Sz(I)$, since we know that $Sz(I)$ has to be 2. Then we need to add 2 everytime we find an application in the definition of virtual redex. Moreover, since we want that $V_x(t) \leq Sz(A \multimap B)$ if $x : A \multimap B$, then we have also to put $Sz(A \multimap B) = Sz(A) + Sz(B) + 2$. Indeed, consider $t \equiv x(y_1)(y_2)(y_3)(y_4)$ where $x : I \multimap (I \multimap (I \multimap (I \multimap I)))$. Then $V_x(t) = 4 \cdot 4 = 16$ and $Sz(I \multimap (I \multimap (I \multimap (I \multimap I)))) = 18$ with the above definition. Instead the size of this type would be only 12 with the usual $sz(A \multimap B) = sz(A) + sz(B) + 1$. Hence, our change of type size is needed.

- Why we put $Sz(A \otimes B) \equiv Sz(A) + Sz(B) + 2$.

  If you consider $t \equiv \texttt{let } w \texttt{ be } y \otimes z \texttt{ in } y \otimes z$ then

$$\begin{aligned}
&\nu(t[\texttt{let } x \texttt{ be } \bullet \texttt{ in } \bullet \otimes \bullet\,/w]) = \\
&\quad = \nu(\texttt{let } (\texttt{let } x \texttt{ be } \bullet \texttt{ in } \bullet \otimes \bullet) \texttt{ be } y \otimes z \texttt{ in } y \otimes z) \\
&\quad = 1 + \nu(\texttt{let } x \texttt{ be } \bullet \texttt{ in let } \bullet \otimes \bullet \texttt{ be } y \otimes z \texttt{ in } y \otimes z) \\
&\quad = 1 + 1 + \nu(\texttt{let } x \texttt{ be } \bullet \texttt{ in } \bullet \otimes \bullet)
\end{aligned}$$

  Hence we need to put $V_w(t) = V_y(y \otimes z) + V_z(y \otimes z) + 2$ being $V_y(y \otimes z) = V_z(y \otimes z) = 0$. Moreover, in order to prove $V_w(t) \leq Sz(A \otimes B)$ if $w : A \otimes B$ by induction, since $V_y(y \otimes z) \leq Sz(A)$ and $V_z(y \otimes z) \leq Sz(B)$, we need to put $Sz(A \otimes B) \equiv Sz(A) + Sz(B) + 2$.

5.1.3 Counterexample with no commuting reductions

Note that in the absence of commuting reductions we can not bound the maximum number of reduction steps for terms under substitutions. Indeed we show here a counterexample.

Proposition 5.19 *The following term in DILL$_{-!}$*

$$x^{I \multimap I}(y^I) : I \ [x : I \multimap I, y : I]$$

has got no reduction bound for all possible substitutions.

Proof. We define the following family of terms in normal form by induction on natural numbers:

$$\begin{aligned}
&t_0 \equiv \texttt{let } w_0 \texttt{ be } \bullet \texttt{ in } \bullet : I \ [w : I] \\
&t_{n+1} \equiv (\texttt{let } w_n \texttt{ be } \bullet \texttt{ in } \lambda w_n.t_n)(\bullet) : I \ [w : I]
\end{aligned}$$

Hence we consider the following term

$$x(y)[\lambda w_n.t_n/x, \bullet/y]$$

We can now prove the following:

$$\nu((\lambda w_n.t_n)(\bullet)) \geq 2 + \nu(\lambda w_{n-1}.t_{n-1})$$

by induction on the natural numbers. Hence

$$\nu(x(y)[\lambda w_n.t_n/x, \bullet/y]) = \nu((\lambda w_n.t_n)(\bullet)) \geq 2(n+1)$$

■

5.1.4 The pure implicational fragment

If we consider the pure implicational fragment of intuitionistic linear logic, that is the lambda calculus with only terms of type $A \multimap B$, then we can provide a tighter bound for the maximum number of steps to normalize a term under substitutions. The only reduction we consider is

$$(\lambda z\colon A.m)(n) \underset{1}{\longrightarrow} m[n/z]$$

By keeping the usual definition of type size

$$sz(A \multimap B) = sz(A) + sz(B) + 1$$

we can define coefficients for virtual redexes as follows.

Def. 5.20 *For every term t of the pure implicational fragment we define $V^i_x(t)$ by induction on the size of t*

$$V^i_x(x) \equiv 0 \qquad V^i_x(y) \equiv 0$$
$$V^i_x(\lambda y.b) \equiv V^i_x(b)$$
$$V^i_x(f(b)) \equiv \begin{cases} V^i_x(f) & \text{if } x \text{ appears in } f \text{ but not in head position} \\ V^i_x(b) & \text{if } x \text{ appears in } b \\ \Sigma_{i=1}^{n+1}(sz(B_i)+1) & \text{if } f \equiv x(b_1)(b_2)\ldots(b_n),\ b \equiv b_{n+1} \\ & \text{where } b_i : B_i \text{ for } i = 1,\ldots,n+1. \end{cases}$$

Then we can prove:

Theorem 5.21 *For every term t in the pure implicational fragment of $DILL_{-!}$ where $\overline{x} \equiv x_1,\ldots,x_n$ are the free variables appearing in t then*

$$\nu(t[\overline{s}/\overline{x}]) \leq \Sigma_{i=0}^{n}\nu(s_i) + \Sigma_{i=0}^{n} V^i_{x_i}(\mathsf{nf}(t)) + \nu(t)$$

The proof is analogous to that of th.5.18 after proving corresponding propositions, lemmas and corollaries regarding the above definition of $V^i_x(t)$.

5.1.5 Extension with primitive recursion

We call $DILL_{pr-!}$ the fragment of $DILL_{pr}$ without the modality !.

Def. 5.22 *The reductions in $DILL_{pr-!}$ are obtained by taking the one step reductions of $DILL_{-!}$ plus the following:*

$$\begin{array}{rcl} \mathsf{Rec}_{\mathsf{l}}(a,g,\epsilon) & \underset{1}{\longrightarrow}^r & a \\ \mathsf{Rec}_{\mathsf{l}}(a,g,\mathsf{cons}(s,c)) & \underset{1}{\longrightarrow}^r & g(\mathsf{Rec}_{\mathsf{l}}(a,g,s),c) \end{array}$$

where we indicate the one step reduction in $DILL_{pr-!}$ with $\underset{1}{\longrightarrow}^r$. Moreover we define $\underset{1}{\longrightarrow}^w$ the one step reduction in $DILL_{pr-!}$ corresponding to those specific to $DILL_{-!}$ to which we add the following:

$$\begin{array}{lcll} \mathsf{Rec}_{\mathsf{l}}(a,g,b) & \underset{1}{\longrightarrow}^r & \mathsf{Rec}_{\mathsf{l}}(a',g,b) & \text{if } a \underset{1}{\longrightarrow}^w a' \\ \mathsf{Rec}_{\mathsf{l}}(a,g,b) & \underset{1}{\longrightarrow}^r & \mathsf{Rec}_{\mathsf{l}}(a,g,b') & \text{if } b \underset{1}{\longrightarrow}^w b' \end{array}$$

We call $\underset{1}{\longrightarrow}^w$ weak reduction step.

Def. 5.23 *For every term t in $DILL_{pr-!}$ we define $\mathsf{wnf}(t)$ as the weak normal form of t in $DILL_{pr-!}$ with respect to $\underset{1}{\longrightarrow}^w$.*

Of course the weak normal forms of $DILL_{pr-!}$ include

$$\mathsf{cons}(s,c) \qquad \mathsf{Rec}_{\mathsf{l}}(a,g,b)$$

where s, c, a, g, b are all in weak normal form.

Def. 5.24 *For every term t in $DILL_{pr-!}$ we define* $\mathsf{rnf}(t)$ *as the normal form in $DILL_{pr-!}$ with respect to* $\underset{1}{\longrightarrow}^r$.

Def. 5.25 *For every term t in $DILL_{pr-!}$ we define*

$\nu_w(t) =$ *the length of the longest reduction sequence of the strongly normalising term t w.r.t.* $\underset{1}{\longrightarrow}^w$

$\nu_r(t) =$ *the length of the longest reduction sequence of the strongly normalising term t w.r.t.* $\underset{1}{\longrightarrow}^r$

Then we can prove

Theorem 5.26 *For every term t in $DILL_{pr-!}$ where $\overline{x} \equiv x_1, \ldots, x_n$ are the free variables appearing in t then*

$$\nu_w(t[\overline{s}/\overline{x}]) \leq \Sigma_{i=0}^n \nu_w(s_i) + \Sigma_{i=0}^n V_{x_i}(\mathsf{wnf}(t)) + \nu_w(t)$$

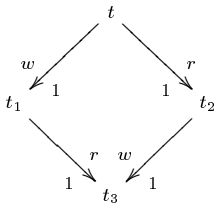
The proof is analogous to that of th.5.18 after proving corresponding propositions, lemmas and corollaries regarding the weak reduction $\underset{1}{\longrightarrow}^w$.
Note that the following proposition holds

Proposition 5.27 *For every term t in $DILL_{pr-!}$ if $t \underset{1}{\longrightarrow}^w t'$ then*

$$\nu_r(t) = 1 + \nu_r(t')$$

This proposition easily follows from the following lemma regarding a Church-Rosser property

Lemma 5.28 *Given $t \underset{1}{\longrightarrow}^w t_1$, $t \underset{1}{\longrightarrow}^r t_2$ and $t_1 \neq t_2$ then there exists a term t_3 such that $t_1 \underset{1}{\longrightarrow}^r t_3$ and $t_2 \underset{1}{\longrightarrow}^w t_3$.*



Then we can deduce

Corollary 5.29 *For every term t in $DILL_{pr-!}$*

$$\nu_r(t) = \nu_w(t) + \nu_r(\mathsf{wnf}(t))$$

By the above propositions we can also prove

Theorem 5.30 *For every term t in $DILL_{pr-!}$ where $\overline{x} \equiv x_1, \ldots, x_n$ are the free variables appearing in t then*

$$\nu_r(t[\overline{s}/\overline{x}]) \leq \Sigma_{i=0}^n \nu_w(s_i) + \Sigma_{i=0}^n V_{x_i}(\mathsf{wnf}(t)) + \nu_w(t) + \nu_r(wnf(t[\overline{s}/\overline{x}]))$$

This is enough to confirm that there is no diagonal which duplicates redexes in variable positions. Indeed, suppose that the term t in normal form has got a free variable x where substituted redexes can be duplicated during its reduction. Then we can find a term $u_n : U$ such that $\nu_r(u_n) = n$, by putting

$$u_0 \equiv x \qquad u_{n+1} \equiv (\lambda y_n.y_n)(u_n)$$

and $n > V_x(t)$. Note that $\mathsf{wnf}(t) = \mathsf{wnf}(t[u_n/x])$ by prop. 5.27 and $\mathsf{wnf}(u_n) = x$. Then we have that $\nu_r(t[u_n/x]) \leq \nu_w(u_n) + V_x(t) + \nu_r(\mathsf{wnf}(t[u_n/x])) < 2n$ against the assumption that $\nu_r(t) \geq 2n$.

Remark 5.31 *Note that the propositions proved so far are also valid with the definition of linear natural numbers given by Paré and Roman [PR89]. In particular, no duplication of redexes of substitutable subterms are possible by taking the linear natural numbers to be $List(I)$ with general elimination rule for lists. This is another confirmation that the semantical contraction defined does not produce duplication of redexes for substitutable subterms.*

Note that the following statement: For every term t in $DILL_{pr-!}$ where $\overline{x} \equiv x_1, \ldots, x_n$ are the free variables appearing in t then

$$\nu_r(t[\overline{s}/\overline{x}]) \leq \Sigma_{i=0}^{n} \nu_r(s_i) + \Sigma_{i=0}^{n} V_{x_i}(\mathsf{wnf}(t)) + \nu_r(t)$$

does not hold. The counterexample can be obtained as follows: take

$$g(x_1) \equiv \mathsf{cons}(\epsilon, \lambda y.x_1(\mathsf{cons}(\mathsf{cons}^n(y), \bullet)))$$

where $\mathsf{cons}^n(\mathsf{cons}^{n-1}(y), \bullet)$ and

$$g(x_1) \ : List(List(\top) \multimap List(\top)) \ [x_1 : List(\top) \multimap List(\top)]$$

then

$$\nu_r(g(x_1)[\lambda w.\mathsf{Rec}_!(\epsilon, \mathsf{cons}, w)/x_1]) \ > \ \nu_r(\lambda w.\mathsf{Rec}_!(\epsilon, \mathsf{cons}, w)) + V_{x_1}(g(x_1))$$

since

$$\nu_r(g(x_1)[\lambda w.\mathsf{Rec}_!(\epsilon, \mathsf{cons}, w)/x_1]) \geq n$$

while $\nu_r(\lambda w.\mathsf{Rec}_!(\epsilon, \mathsf{cons}, w)) = 0$ and $V_{x_1}(g(x_1)) = Sz(List(\top)) + 2$. If we take $n > Sz(List(\top)) + 2$ we get the counterexample.
We can establish some tighter bound in some cases. For example, we can prove by induction on the size of $\mathsf{Rec}_!$

Proposition 5.32 *For every term g in $DILL_{pr-!}$ where $x_1 : List(\top), x_2 : \top$ are the free variables appearing in g such that*

$$\nu_r(g[s_1/x_1, s_2/x_2]) \leq \Sigma_{i=0}^{n} \nu_r(s_i) + \Sigma_{i=1}^{2} V_{x_i}(\mathsf{wnf}(g)) + \nu_r(g)$$

then

$$\nu_r(\mathsf{Rec}_!(a, g, b)) \leq \nu_r(a) + \nu_r(b) + size(\mathsf{rnf}(b))(\Sigma_{i=1}^{2} V_{x_i}(\mathsf{wnf}(g)) + \nu_r(g) + 1)$$

Moreover we can also prove

Proposition 5.33 *For every term $g : List(\top)$ in $DILL_{pr-!}$ containing no $\mathsf{Rec}_!$-expression, where $x_1 : List(\top), x_2 : \top$ are the free variables appearing in g, then*

$$\nu_r(g[s_1/x_1, s_2/x_2]) \leq \nu_r(s_1) + \nu_r(s_2) + \Sigma_{i=1}^{2} V_{x_i}(\mathsf{wnf}(g)) + \nu_r(g)$$

Hence combining the above propositions we get

Corollary 5.34 *For every term g in $DILL_{pr-!}$ containing no $\mathsf{Rec}_!$-expression*

$$\nu_r(\mathsf{Rec}_!(a, g, b)) \leq \nu_r(a) + \nu_r(b) + size(\mathsf{rnf}(b))(\Sigma_{i=1}^{2} V_{x_i}(\mathsf{wnf}(g)) + \nu_r(g) + 1)$$

Note that to get this corollary the restriction on the elimination rule for lists is crucial as you can see from the above counterexample.

6 Conclusion

Here, we have sorted out a notion of linear primitive recursion to form a Linear Natural Numbers Object that does not satisfy weakening and contraction contrary to Parè-Roman's LNNO. This let us improve the decoration of functional programs into the linear setting in order to make their reductions more efficient. We have also established some bounds to normalize terms under substitution towards an understanding of the these linear systems from a complexity point of view.

References

[Bar97] A. Barber. *Linear Type Theories, Semantics and Action Calculi.* PhD thesis, University of Edinburgh, 1997.

[BBdH93] N. Benton, G. Bierman, V. de Paiva, and M. Hyland. A term calculus for intuitionistic linear logic? In *Proc. of Typed Lambda Calculus and Applications.*, volume 664 of *Lecture Notes in Computer Science*, pages 75–90. Springer Verlag, 1993.

[Ben95] Nick Benton. A mixed linear and non-linear logic: Proofs, terms and models. In *Proceedings of Computer Science Logic '94, Kazimierz, Poland.* Lecture Notes in Computer Science No. 933, Berlin, Heidelberg, New York, 1995.

[Bie94] G. Bierman. What is a categorical model of intuitionistic linear logic? In *Proc. of the Second International Conference on Typed Lambda Calculus and Applications.*, volume 902 of *Lecture Notes in Computer Science.* Springer Verlag, 1994.

[BP97] A. Barber and G. Plotkin. Dual intuitionistic linear logic. Technical report, LFCS, University of Edinburgh, 1997.

[Bra94] T. Braüner. The Girard translation extended with recursion. In *Proc. of CSL '94*, volume 933 of *Lecture Notes in Computer Science.* Springer Verlag, 1994.

[Gha95] N. Ghani. *Adjoint Rewriting.* PhD thesis, University of Edinburgh, 1995. Published as CST-122-95 and ECS-LFCS-95-339.

[Gir87] J.Y. Girard. Linear logic. *Theoretical Computer Science*, 50:1–101, 1987.

[GTL89] J.Y. Girard, P. Taylor, and Y. Lafont. *Proofs and types.* Number 7 in Cambridge Tracts in Theoretical Computer Science. Cambridge University Press, 1989.

[Has99] M. Hasegawa. *Models of sharing graphs. A categorical semantics of let and letrec.* PhD thesis, University of Edinburgh, 1999. PCPHC/BCS Distinguished Dissertations. Springer-Verlag London.

[Hof99] M. Hofmann. Linear types and non-size-increasing polynomial time computation. In *Proc. of LICS '99*, IEEE, Computer Society Press. Springer Verlag, 1999.

[Plo77] G. Plotkin. LCF considered as a programming language. *Theoretical Computer Science*, 5:223–255, 1977.

[PR89] R. Parè and L. Roman. Monoidal categories with natural numbers object. *Studia Logica*, 3:361–3761, 1989.